

**BỘ GIÁO DỤC VÀ ĐÀO TẠO**

**TRƯỜNG ĐẠI HỌC ĐẠI NAM**



**PHÙNG HỮU TÀI - 1771040021**

**NGUYỄN VIỆT CHUNG - 1771040005**

**ĐẶNG QUỐC AN - 1675020001**

**NGUYỄN ĐOÀN NGỌC LINH - 1771040016**

**PHÂN LOẠI RÁC THẢI BẰNG ẢNH PHỤC VỤ MÔI TRƯỜNG XANH**

**BÀI TẬP LỚN HỌC PHẦN HỌC SÂU**

**NGÀNH: CÔNG NGHỆ THÔNG TIN**

**HÀ NỘI, NĂM 2026**

**BỘ GIÁO DỤC VÀ ĐÀO TẠO**

**TRƯỜNG ĐẠI HỌC ĐẠI NAM**



**PHÙNG HỮU TÀI - 1771040021**

**NGUYỄN VIỆT CHUNG - 1771040005**

**ĐẶNG QUỐC AN - 1675020001**

**NGUYỄN ĐOÀN NGỌC LINH - 1771040016**

**PHÂN LOẠI RÁC THẢI BẰNG ẢNH PHỤC VỤ MÔI TRƯỜNG XANH**

**BÀI TẬP LỚN HỌC PHẦN HỌC SÂU**

**NGÀNH: CÔNG NGHỆ THÔNG TIN**

**CHUYÊN NGÀNH: KHOA HỌC MÁY TÍNH**

**GIẢNG VIÊN HƯỚNG DẪN: ThS. Lê Thị Thùy Trang**

**HÀ NỘI, NĂM 2026**

## LỜI CAM ĐOAN

Nhóm chúng em xin cam đoan bài tập lớn với đề tài "Phân loại rác thải bằng ảnh phục vụ môi trường xanh" là kết quả nghiên cứu, tìm hiểu và thực hiện của chính nhóm trong quá trình học tập học phần Học Sâu (CSC4005), dưới sự hướng dẫn của ThS. Lê Thị Thùy Trang.

Toàn bộ mã nguồn, mô hình huấn luyện, kết quả thí nghiệm trên Weights & Biases, file ONNX và demo Streamlit được nhóm tự xây dựng, không sao chép từ bất kỳ nguồn nào khác mà không có sự trích dẫn hợp lệ. Bộ dữ liệu Garbage Classification được sử dụng trong báo cáo là dữ liệu công khai trên Kaggle, được trích dẫn rõ nguồn trong phần tài liệu tham khảo.

Các số liệu, hình ảnh, bảng biểu, tài liệu và nội dung trình bày trong báo cáo là trung thực, có nguồn gốc rõ ràng và được sử dụng đúng mục đích học tập. Nhóm xin chịu trách nhiệm về tính chính xác và trung thực của toàn bộ nội dung báo cáo.

*Hà Nội, ngày 30 tháng 6 năm 2026*

**Nhóm thực hiện**

**PHÙNG HỮU TÀI - 1771040021**  
**NGUYỄN VIỆT CHUNG - 1771040005**  
**ĐẶNG QUỐC AN - 1675020001**  
**NGUYỄN ĐOÀN NGỌC LINH -**  
**1771040016**

## LỜI CẢM ƠN

Nhóm chúng em xin gửi lời cảm ơn chân thành tới Ban Giám hiệu Trường Đại học Đại Nam, Khoa Công nghệ thông tin, cùng các thầy cô đã tạo điều kiện thuận lợi cho nhóm trong suốt quá trình học tập và thực hiện học phần Học Sâu (CSC4005).

Đặc biệt, nhóm xin gửi lời cảm ơn sâu sắc đến ThS. Lê Thị Thùy Trang — giảng viên hướng dẫn đã tận tình truyền đạt kiến thức, cung cấp tài liệu, định hướng phương pháp thực hiện và góp ý chi tiết trong suốt quá trình nhóm xây dựng và hoàn thiện bài tập lớn với đề tài ”Phân loại rác thải bằng ảnh phục vụ môi trường xanh”. Những hướng dẫn cụ thể về quy trình xây dựng pipeline học sâu, cách sử dụng Weights & Biases để theo dõi thí nghiệm, yêu cầu triển khai ONNX và model compression đã giúp nhóm hình thành tư duy làm việc khoa học, có hệ thống và bài bản hơn.

Trong quá trình thực hiện, nhóm cũng xin cảm ơn sự hỗ trợ và chia sẻ kinh nghiệm từ các bạn cùng lớp, đã cùng nhóm thảo luận, trao đổi về cách xử lý dữ liệu, lựa chọn kiến trúc mô hình và khắc phục các lỗi kỹ thuật phát sinh trong quá trình huấn luyện trên môi trường không có GPU.

Mặc dù đã cố gắng hoàn thiện báo cáo và sản phẩm một cách tốt nhất, song do kiến thức và kinh nghiệm thực tế còn hạn chế, đặc biệt là giới hạn về tài nguyên tính toán, báo cáo khó tránh khỏi những thiếu sót. Nhóm kính mong nhận được ý kiến góp ý của quý thầy cô để báo cáo và sản phẩm được hoàn thiện hơn trong các bài tập tiếp theo.

*Hà Nội, ngày 30 tháng 6 năm 2026*

**Nhóm thực hiện**

**PHÙNG HỮU TÀI - 1771040021**  
**NGUYỄN VIỆT CHUNG - 1771040005**  
**ĐẶNG QUỐC AN - 1675020001**  
**NGUYỄN ĐOÀN NGỌC LINH -**  
**1771040016**

## KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT

| TỪ VIẾT TẮT | TÊN ĐẦY ĐỦ CỦA TỪ VIẾT TẮT                                                                              |
|-------------|---------------------------------------------------------------------------------------------------------|
| AI          | Artificial Intelligence (Trí tuệ nhân tạo)                                                              |
| CNN         | Convolutional Neural Network (Mạng nơ-ron tích chập)                                                    |
| CPU         | Central Processing Unit (Bộ xử lý trung tâm)                                                            |
| GPU         | Graphics Processing Unit (Bộ xử lý đồ họa)                                                              |
| NAS         | Neural Architecture Search (Tìm kiếm kiến trúc mạng nơ-ron)                                             |
| ONNX        | Open Neural Network Exchange (Định dạng trao đổi mô hình mạng nơ-ron mở)                                |
| ReLU        | Rectified Linear Unit (Hàm kích hoạt tuyến tính chỉnh lưu)                                              |
| W&B         | Weights & Biases (Công cụ theo dõi thí nghiệm học máy)                                                  |
| F1-score    | Harmonic mean of Precision and Recall (Trung bình điều hòa của Precision và Recall)                     |
| FLOPs       | Floating Point Operations (Số phép tính dấu chấm động)                                                  |
| SGD         | Stochastic Gradient Descent (Giảm gradient ngẫu nhiên)                                                  |
| Adam/AdamW  | Adaptive Moment Estimation (with Weight decay) (Thuật toán tối ưu thích nghi có/không phân rã trọng số) |
| KD          | Knowledge Distillation (Chắt lọc kiến thức)                                                             |
| Grad-CAM    | Gradient-weighted Class Activation Mapping (Bản đồ kích hoạt lớp theo trọng số gradient)                |
| fps / img/s | Frames per second / Images per second (Số khung hình/ảnh xử lý mỗi giây)                                |

## DANH MỤC CÁC BẢNG, SƠ ĐỒ, HÌNH

### BẢNG:

| Tên bảng                                                     | Trang |
|--------------------------------------------------------------|-------|
| Bảng 1.1. Phân bố dữ liệu theo lớp .....                     | 4     |
| Bảng 2.1. Cấu hình hyperparameter huấn luyện .....           | 11    |
| Bảng 3.1. Tổng hợp kết quả 5 runs .....                      | 14    |
| Bảng 3.2. Thống kê mẫu dự đoán sai của EfficientNet B0 ..... | 20    |
| Bảng 4.1. So sánh PyTorch và ONNX Runtime .....              | 25    |

### SƠ ĐỒ:

| Tên sơ đồ                                       | Trang |
|-------------------------------------------------|-------|
| Sơ đồ 2.1. Kiến trúc mô hình CNN Baseline ..... | 9     |

### HÌNH:

| <b>Tên hình</b>                                                                            | <b>Trang</b> |
|--------------------------------------------------------------------------------------------|--------------|
| Hình 1.1. Mẫu ảnh đại diện của 6 lớp rác thải . . . . .                                    | 4            |
| Hình 1.2. Phân bố số lượng ảnh theo lớp rác thải . . . . .                                 | 5            |
| Hình 2.1. Kiến trúc EfficientNet B0 (minh họa compound scaling) . . . . .                  | 10           |
| Hình 2.2. W&B Dashboard tổng hợp 5 runs thí nghiệm . . . . .                               | 13           |
| Hình 3.1. Learning curves của CNN Baseline (20 epochs) . . . . .                           | 15           |
| Hình 3.2. Learning curves của ResNet18 Finetune (10 epochs) . . . . .                      | 15           |
| Hình 3.3. Learning curves của MobileNetV2 Finetune (10 epochs) . . . . .                   | 16           |
| Hình 3.4. Learning curves của EfficientNet B0 - Best Model (10 epochs) . . .               | 16           |
| Hình 3.5. Confusion Matrix của CNN Baseline (test set, 758 mẫu) . . . . .                  | 17           |
| Hình 3.6. Confusion Matrix của EfficientNet B0 - Best Model (test set, 758 mẫu) . . . . .  | 18           |
| Hình 3.7. Các mẫu dự đoán sai tiêu biểu của EfficientNet B0 . . . . .                      | 21           |
| Hình 3.8. Grad-CAM visualization: Original   Heatmap   Overlay (EfficientNet B0) . . . . . | 23           |
| Hình 4.1. Giao diện Demo Streamlit - Hệ thống phân loại rác thải . . . . .                 | 26           |
| Hình .2. Ảnh minh họa bổ sung quá trình thực hiện . . . . .                                | 30           |

## MỤC LỤC

|                                                                     |            |
|---------------------------------------------------------------------|------------|
| <b>LỜI CAM ĐOAN</b> . . . . .                                       | <b>i</b>   |
| <b>LỜI CẢM ƠN</b> . . . . .                                         | <b>ii</b>  |
| <b>KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT</b> . . . . .                        | <b>iii</b> |
| <b>DANH MỤC CÁC BẢNG, SƠ ĐỒ, HÌNH</b> . . . . .                     | <b>iv</b>  |
| <b>LỜI MỞ ĐẦU</b> . . . . .                                         | <b>vi</b>  |
| <b>CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN VÀ DỮ LIỆU</b> . . . . .           | <b>1</b>   |
| <b>1.1. Bối cảnh và tính cấp thiết</b> . . . . .                    | <b>1</b>   |
| <b>1.2. Mục tiêu và phạm vi</b> . . . . .                           | <b>2</b>   |
| <b>1.3. Bộ dữ liệu Garbage Classification</b> . . . . .             | <b>3</b>   |
| <b>1.4. Phân tích dữ liệu</b> . . . . .                             | <b>4</b>   |
| <b>1.5. Chia tập dữ liệu</b> . . . . .                              | <b>5</b>   |
| <b>1.6. Tiền xử lý và tăng cường dữ liệu</b> . . . . .              | <b>6</b>   |
| <b>CHƯƠNG 2. MÔ HÌNH HỌC SÂU VÀ THIẾT LẬP THÍ NGHIỆM</b> . . . . .  | <b>8</b>   |
| <b>2.1. Tổng quan kiến trúc</b> . . . . .                           | <b>8</b>   |
| <b>2.2. CNN Baseline from Scratch</b> . . . . .                     | <b>8</b>   |
| <b>2.3. Transfer Learning và Fine-tuning</b> . . . . .              | <b>9</b>   |
| 2.3.1. <i>ResNet18</i> . . . . .                                    | 9          |
| 2.3.2. <i>MobileNetV2</i> . . . . .                                 | 10         |
| 2.3.3. <i>EfficientNet B0</i> . . . . .                             | 10         |
| <b>2.4. Cấu hình huấn luyện</b> . . . . .                           | <b>11</b>  |
| <b>2.5. Môi trường thực nghiệm</b> . . . . .                        | <b>12</b>  |
| <b>2.6. Theo dõi thí nghiệm bằng Weights &amp; Biases</b> . . . . . | <b>12</b>  |
| <b>2.7. Chiến lược chọn best model</b> . . . . .                    | <b>13</b>  |
| <b>CHƯƠNG 3. KẾT QUẢ, PHÂN TÍCH VÀ PHÂN TÍCH LỖI</b> . . . . .      | <b>14</b>  |
| <b>3.1. Bảng tổng hợp kết quả</b> . . . . .                         | <b>14</b>  |
| <b>3.2. Phân tích learning curves</b> . . . . .                     | <b>14</b>  |
| 3.2.1. <i>CNN Baseline</i> . . . . .                                | 14         |
| 3.2.2. <i>ResNet18 Finetune</i> . . . . .                           | 15         |
| 3.2.3. <i>MobileNetV2 Finetune</i> . . . . .                        | 15         |
| 3.2.4. <i>EfficientNet B0 (Best Model)</i> . . . . .                | 16         |
| <b>3.3. Phân tích confusion matrix</b> . . . . .                    | <b>17</b>  |
| 3.3.1. <i>CNN Baseline</i> . . . . .                                | 17         |



|                                                        |           |
|--------------------------------------------------------|-----------|
| 3.3.2. <i>EfficientNet B0 (Best Model)</i> . . . . .   | 18        |
| <b>3.4. So sánh các mô hình</b> . . . . .              | <b>18</b> |
| <b>3.5. Thống kê mẫu dự đoán sai</b> . . . . .         | <b>19</b> |
| <b>3.6. Phân tích chi tiết từng loại lỗi</b> . . . . . | <b>21</b> |
| <b>3.7. Grad-CAM Visualization</b> . . . . .           | <b>23</b> |
| <b>CHƯƠNG 4. TRIỂN KHAI ONNX VÀ DEMO</b> . . . . .     | <b>24</b> |
| 4.1. Export sang ONNX . . . . .                        | 24        |
| 4.2. Benchmark và so sánh . . . . .                    | 24        |
| 4.3. Demo Streamlit . . . . .                          | 25        |
| <b>KẾT LUẬN</b> . . . . .                              | <b>27</b> |
| <b>PHỤ LỤC</b> . . . . .                               | <b>29</b> |
| <b>DANH MỤC TÀI LIỆU THAM KHẢO</b> . . . . .           | <b>32</b> |

## LỜI MỞ ĐẦU

Trong bối cảnh ô nhiễm môi trường ngày càng trở nên nghiêm trọng trên toàn cầu, việc phân loại và xử lý rác thải đúng cách là một trong những giải pháp quan trọng để bảo vệ môi trường sống. Theo báo cáo của Ngân hàng Thế giới, mỗi năm thế giới thải ra khoảng 2 tỷ tấn rác sinh hoạt, trong đó chỉ có khoảng 13,5% được tái chế. Tại Việt Nam, lượng rác thải sinh hoạt phát sinh khoảng 64.000 tấn mỗi ngày, nhưng tỉ lệ tái chế còn rất thấp, một phần nguyên nhân là do công tác phân loại rác tại nguồn chưa được thực hiện hiệu quả và đồng bộ. Việc phân loại rác thải thủ công đòi hỏi nhiều nhân lực, thời gian và dễ xảy ra sai sót, trong khi đó các bãi chôn lấp ngày càng quá tải và chi phí xử lý rác ngày càng tăng cao.

Sự phát triển mạnh mẽ của trí tuệ nhân tạo và học sâu trong những năm gần đây đã mở ra hướng đi mới đầy tiềm năng: tự động hóa quá trình phân loại rác thải thông qua nhận diện hình ảnh. Các mạng nơ-ron tích chập (CNN) và các kiến trúc hiện đại như ResNet, MobileNet, EfficientNet đã chứng minh khả năng vượt trội trong các bài toán nhận diện và phân loại ảnh, mở ra cơ hội xây dựng các hệ thống thông minh có thể phân loại rác thải với độ chính xác cao và tốc độ xử lý nhanh, phù hợp triển khai trong các băng chuyền thu gom rác công nghiệp, thùng rác thông minh tại khu đô thị, hoặc ứng dụng di động hỗ trợ người dùng phân loại rác ngay tại nhà.

Báo cáo này trình bày toàn bộ quá trình thực hiện bài tập lớn học phần Học Sâu (CSC4005) với đề tài **”Phân loại rác thải bằng ảnh phục vụ môi trường xanh”**. Nhóm đã xây dựng một pipeline học sâu hoàn chỉnh, bao gồm: thu thập và phân tích bộ dữ liệu Garbage Classification gồm 2.527 ảnh thuộc 6 lớp rác thải; thiết kế và huấn luyện một mô hình CNN baseline từ đầu; áp dụng kỹ thuật transfer learning với ba kiến trúc pretrained hiện đại (ResNet18, MobileNetV2, EfficientNet B0); theo dõi và quản lý toàn bộ quá trình thí nghiệm bằng công cụ Weights & Biases với 5 runs có log đầy đủ; phân tích sâu kết quả qua learning curves, confusion matrix và phân tích lỗi định tính trên các mẫu dự đoán sai thực tế; và cuối cùng triển khai mô hình tốt nhất sang định dạng ONNX, benchmark hiệu năng, đồng thời xây dựng một demo web tương tác bằng Streamlit.

Trong quá trình thực hiện, nhóm đã học được nhiều kiến thức quý báu về thiết kế thực nghiệm khoa học, cách sử dụng các công cụ quản lý thí nghiệm hiện đại, kỹ năng phân tích kết quả định lượng kết hợp với phân tích lỗi định tính, và quy trình đưa một mô hình học sâu từ môi trường nghiên cứu đến khả năng triển khai thực tế. Đây là nền tảng quan trọng để nhóm có thể áp dụng học sâu vào các bài toán thực tiễn khác trong tương lai, đặc biệt là các bài toán có ý nghĩa với môi trường và phát triển bền vững.

Nhóm xin chân thành cảm ơn ThS. Lê Thị Thùy Trang đã tận tình hướng dẫn, cung

cấp tài liệu và định hướng cho nhóm trong suốt quá trình thực hiện bài tập lớn. Những hướng dẫn chi tiết về quy trình xây dựng pipeline học sâu, cách sử dụng W&B để theo dõi thí nghiệm, và yêu cầu triển khai ONNX đã giúp nhóm hình thành tư duy làm việc khoa học và có hệ thống. Mặc dù đã cố gắng hết sức trong thời gian thực hiện, báo cáo chắc chắn còn nhiều thiếu sót do hạn chế về thời gian và tài nguyên tính toán (máy không có GPU), nhóm rất mong nhận được sự góp ý của cô để hoàn thiện hơn trong các bài tập tiếp theo.

## Chương 1

# GIỚI THIỆU BÀI TOÁN VÀ DỮ LIỆU

### 1.1 Bối cảnh và tính cấp thiết

Rác thải là một trong những vấn đề môi trường cấp bách nhất của thế kỷ 21. Sự gia tăng dân số, đô thị hóa nhanh chóng và thay đổi thói quen tiêu dùng đã khiến lượng rác thải sinh hoạt phát sinh ngày càng lớn, vượt quá khả năng xử lý của nhiều địa phương. Theo báo cáo của Ngân hàng Thế giới, mỗi năm thế giới thải ra khoảng 2 tỷ tấn rác sinh hoạt, trong đó chỉ có khoảng 13,5% được tái chế, phần còn lại bị chôn lấp hoặc đốt, gây ra những tác động tiêu cực lâu dài đến môi trường đất, nước và không khí.

Tại Việt Nam, lượng rác thải sinh hoạt phát sinh khoảng 64.000 tấn mỗi ngày, tập trung chủ yếu tại các thành phố lớn như Hà Nội và Thành phố Hồ Chí Minh. Tuy nhiên, tỉ lệ tái chế còn rất thấp, một phần nguyên nhân quan trọng là do công tác phân loại rác tại nguồn chưa được thực hiện hiệu quả và đồng bộ trên quy mô lớn. Nhiều chương trình phân loại rác tại nguồn đã được triển khai thử nghiệm nhưng gặp khó khăn do thói quen của người dân, thiếu cơ sở hạ tầng thu gom riêng biệt, và đặc biệt là việc thiếu công cụ hỗ trợ phân loại nhanh, chính xác ngay tại điểm phát sinh rác.

Phân loại rác thải đúng cách là bước đầu tiên và quan trọng nhất trong toàn bộ chuỗi xử lý rác. Khi rác được phân loại tốt ngay từ nguồn, tỉ lệ tái chế tăng lên đáng kể, giảm áp lực cho các bãi chôn lấp, giảm chi phí xử lý và góp phần giảm phát thải khí nhà kính từ quá trình phân hủy rác hữu cơ lẫn với rác vô cơ. Tuy nhiên, việc phân loại thủ công đòi hỏi lao động lớn, tốn thời gian và không đảm bảo tính nhất quán giữa các cá nhân thực hiện — một người có thể phân loại một vật thể vào lớp này, người khác lại phân loại vào lớp khác do thiếu tiêu chuẩn rõ ràng hoặc thiếu kinh nghiệm.

Học sâu, đặc biệt là các mạng nơ-ron tích chập (Convolutional Neural Network - CNN) và các kiến trúc hiện đại như EfficientNet, MobileNet, ResNet, đã chứng minh khả năng vượt trội trong các bài toán nhận diện và phân loại ảnh trong gần một thập kỷ qua, đạt được độ chính xác vượt qua khả năng của con người trong nhiều benchmark phân loại ảnh quy mô lớn như ImageNet. Việc ứng dụng học sâu vào phân loại rác thải tự động mở ra cơ hội xây dựng các hệ thống thông minh có khả năng phân loại hàng nghìn vật thể mỗi giờ với độ chính xác cao và tính nhất quán tuyệt đối, hoàn toàn phù hợp để triển khai trong các băng chuyền thu gom rác công nghiệp, thùng rác thông minh tại các khu đô thị, hoặc tích hợp vào ứng dụng di động hỗ trợ người dùng phân loại rác ngay tại nhà.

Đây chính là động lực để nhóm lựa chọn đề tài ”Phân loại rác thải bằng ảnh phục vụ môi trường xanh” làm chủ đề thực hiện bài tập lớn. Bài toán không chỉ có ý nghĩa thực tiễn và xã hội cao mà còn là cơ hội tốt để nhóm áp dụng toàn bộ kiến thức đã được học trong học phần CSC4005, từ việc xây dựng CNN baseline từ đầu, fine-tune các mô hình pretrained trên ImageNet, đến triển khai mô hình dưới dạng sản phẩm có thể sử dụng thực tế thông qua ONNX và demo web.

## 1.2 Mục tiêu và phạm vi

Mục tiêu chính của bài tập lớn là xây dựng một pipeline học sâu hoàn chỉnh, có khả năng phân loại chính xác ảnh rác thải vào một trong 6 lớp được định nghĩa trước, đồng thời tuân thủ đầy đủ quy trình thực nghiệm khoa học theo yêu cầu của học phần. Cụ thể, nhóm đặt ra các mục tiêu chi tiết như sau:

1. Xây dựng và đánh giá một mô hình CNN baseline được huấn luyện từ đầu (train from scratch), làm điểm xuất phát để so sánh với các mô hình phức tạp hơn, từ đó đánh giá định lượng giá trị mà transfer learning mang lại.
2. Áp dụng kỹ thuật transfer learning với tối thiểu ba kiến trúc mạng nơ-ron pretrained khác nhau (ResNet18, MobileNetV2, EfficientNet B0), so sánh hiệu quả giữa các kiến trúc về độ chính xác, số lượng tham số và thời gian huấn luyện.
3. So sánh hai chiến lược huấn luyện: head-only (chỉ train classifier head, đóng băng backbone) và fine-tune (train toàn bộ mô hình), từ đó rút ra kết luận về chiến lược phù hợp với đặc thù bài toán phân loại rác thải.
4. Theo dõi và quản lý toàn bộ quá trình thí nghiệm bằng công cụ Weights & Biases (W&B), đảm bảo có tối thiểu 5 runs được log đầy đủ hyperparameter và metrics, tạo cơ sở minh bạch và có thể tái lập cho việc đánh giá.
5. Đánh giá mô hình bằng các metric phù hợp gồm accuracy và macro F1-score, đồng thời phân tích confusion matrix để hiểu rõ hành vi của mô hình trên từng lớp cụ thể.
6. Thực hiện phân tích lỗi định tính dựa trên tối thiểu 10 mẫu dự đoán sai thực tế, tìm hiểu nguyên nhân sâu xa và đề xuất hướng cải thiện cụ thể.
7. Export mô hình tốt nhất sang định dạng ONNX, kiểm thử suy luận bằng ONNX Runtime, và benchmark so sánh hiệu năng (kích thước, latency, throughput) với mô hình PyTorch gốc.

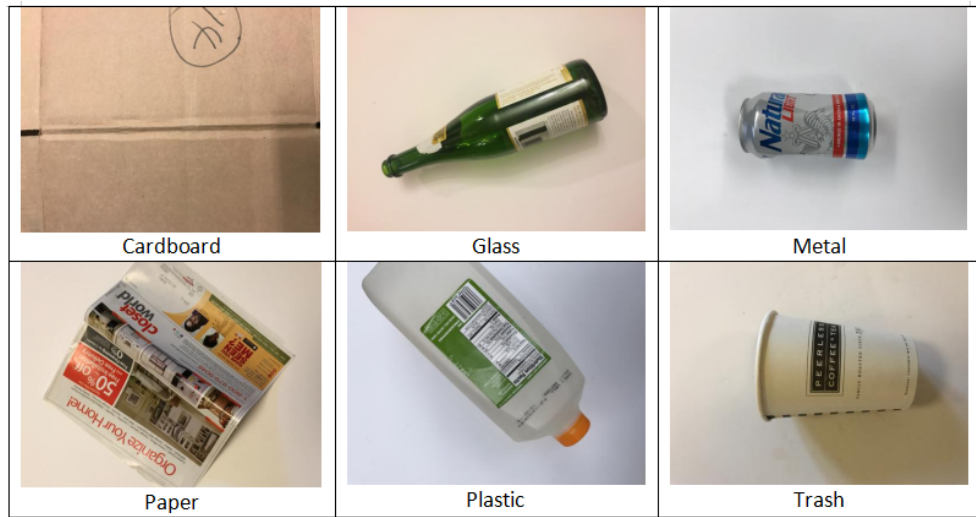
8. Xây dựng một demo web tương tác bằng Streamlit, cho phép người dùng trải nghiệm trực tiếp hệ thống phân loại rác thải với giao diện thân thiện bằng tiếng Việt.

Về phạm vi, bài tập lớn sử dụng bộ dữ liệu Garbage Classification được công bố công khai trên Kaggle, gồm 2.527 ảnh thuộc 6 lớp rác thải phổ biến nhất trong sinh hoạt hàng ngày: cardboard (bìa carton), glass (thủy tinh), metal (kim loại), paper (giấy), plastic (nhựa) và trash (rác thông thường khó tái chế). Đây là bộ dữ liệu ảnh thực tế, được chụp trong điều kiện ánh sáng và góc độ đa dạng, phản ánh khá sát độ khó của bài toán trong môi trường thực tế, khác với các bộ dữ liệu benchmark được chuẩn hóa cao như MNIST hay CIFAR. Phạm vi nghiên cứu không bao gồm việc thu thập dữ liệu mới hay mở rộng số lớp phân loại, mà tập trung vào việc xây dựng, huấn luyện, đánh giá và triển khai mô hình một cách khoa học và có hệ thống trên bộ dữ liệu đã có.

### 1.3 Bộ dữ liệu Garbage Classification

Nhóm sử dụng bộ dữ liệu Garbage Classification được công bố công khai trên Kaggle tại địa chỉ <https://www.kaggle.com/datasets/asdasdasdasdas/garbage-classification>. Đây là bộ dữ liệu ảnh thực tế về rác thải trong môi trường đô thị và sinh hoạt, được thu thập và gán nhãn thủ công với mục đích phục vụ nghiên cứu và giáo dục về phân loại rác thải tự động.

Bộ dữ liệu gồm 2.527 ảnh màu RGB thuộc 6 lớp rác thải phổ biến. Mỗi ảnh được chụp trong điều kiện ánh sáng và góc độ đa dạng, với nền ảnh khác nhau bao gồm nền trắng studio, nền xám, ngoài trời tự nhiên, hoặc trên bàn làm việc, phản ánh sự đa dạng của dữ liệu thực tế mà một hệ thống triển khai thật sẽ phải đối mặt. Kích thước ảnh gốc không đồng nhất, dao động từ khoảng  $300 \times 300$  đến  $1000 \times 1000$  pixel, được nhóm resize đồng bộ về kích thước  $224 \times 224$  pixel trong quá trình tiền xử lý để phù hợp với input của các mô hình pretrained trên ImageNet.



Hình 1.1. Mẫu ảnh đại diện của 6 lớp rác thải

Nguồn: *Garbage Classification Dataset - Kaggle*

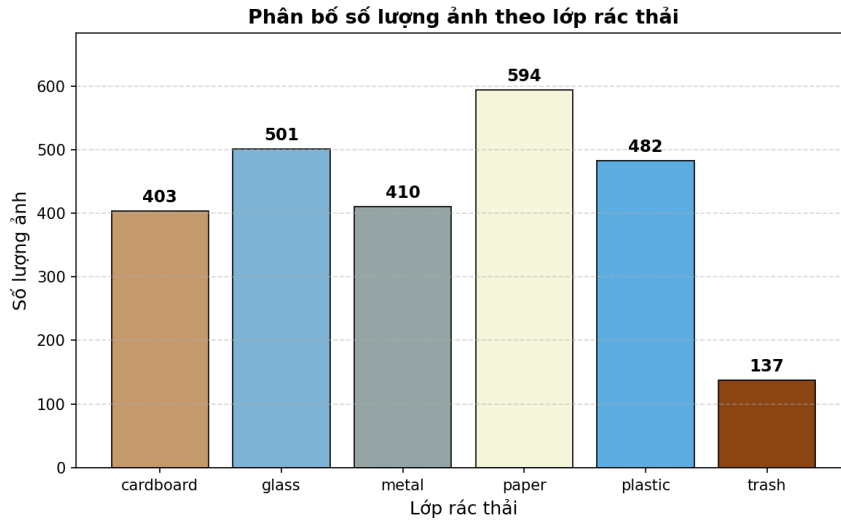
#### 1.4 Phân tích dữ liệu

Phân tích phân bố dữ liệu theo lớp cho thấy sự mất cân bằng đáng kể giữa các lớp, đây là một đặc điểm quan trọng cần lưu ý khi thiết kế thí nghiệm và lựa chọn metric đánh giá. Lớp paper có số lượng ảnh nhiều nhất với 594 ảnh, chiếm 23,5% tổng số dữ liệu, trong khi lớp trash có số lượng ít nhất, chỉ 137 ảnh, chiếm 5,4%. Sự mất cân bằng này có thể ảnh hưởng đến khả năng học của mô hình, đặc biệt với các lớp có ít dữ liệu như trash, khiến mô hình có ít cơ hội học được đặc trưng đặc thù của lớp này một cách đầy đủ và đa dạng.

Bảng 1.1. Phân bố dữ liệu theo lớp

| Lớp         | Tên tiếng Việt   | Số ảnh gốc   | Train (70%)  | Val+Test (30%) |
|-------------|------------------|--------------|--------------|----------------|
| cardboard   | Bìa carton       | 403          | 282          | 121            |
| glass       | Thủy tinh        | 501          | 351          | 150            |
| metal       | Kim loại         | 410          | 287          | 123            |
| paper       | Giấy             | 594          | 416          | 178            |
| plastic     | Nhựa             | 482          | 337          | 145            |
| trash       | Rác thông thường | 137          | 96           | 41             |
| <b>Tổng</b> |                  | <b>2.527</b> | <b>1.769</b> | <b>758</b>     |

Nguồn: Nhóm tổng hợp từ *Garbage Classification Dataset*



Hình 1.2. Phân bố số lượng ảnh theo lớp rác thải

*Nguồn: Nhóm tổng hợp*

Quan sát trực quan các ảnh trong từng lớp, nhóm nhận thấy một số thách thức đặc thù mà mô hình sẽ phải đối mặt. Lớp metal và glass đôi khi có màu sắc và hình dạng tương đồng, đặc biệt là các vật thể dạng chai, lon hình trụ — khi nhìn từ một số góc độ, ánh sáng phản chiếu trên bề mặt kim loại có thể trông tương tự với độ trong và độ phản chiếu của thủy tinh. Lớp paper và cardboard đều có màu nâu hoặc trắng với texture phẳng, khó phân biệt nếu chỉ nhìn một phần ảnh hoặc ảnh có độ phân giải thấp. Lớp trash là lớp tạp nhất vì về bản chất nó chứa các vật thể không phù hợp rõ ràng vào 5 lớp còn lại, ví dụ như rác hỗn hợp, đồ vật bản, hoặc vật liệu composite kết hợp nhiều thành phần. Những quan sát này sẽ được kiểm chứng và thể hiện rõ hơn trong phân phân tích confusion matrix và phân tích lỗi ở Chương 3.

## 1.5 Chia tập dữ liệu

Dữ liệu được chia ngẫu nhiên theo tỉ lệ 70/15/15 cho train/validation/test, sử dụng random seed cố định (seed=42) để đảm bảo khả năng tái lập kết quả giữa các lần chạy lại thí nghiệm. Cách chia ngẫu nhiên này, kết hợp với việc lấy mẫu trên toàn bộ tập dữ liệu, vẫn đảm bảo mỗi lớp đều có đại diện hợp lý trong cả 3 tập do kích thước mẫu tương đối lớn. Kết quả chia tập được lưu lại dưới dạng các file CSV (train.csv, val.csv, test.csv) trong thư mục data/splits/ để có thể theo dõi, kiểm tra và tái sử dụng cho các lần thí nghiệm tiếp theo, đảm bảo tính nhất quán giữa các runs khác nhau.

Cụ thể, sau khi chia, số lượng mẫu của từng tập là:

- Tập train: 1.769 ảnh (70%) — dùng để huấn luyện và cập nhật trọng số mô hình.



- Tập validation: 379 ảnh (15%) — dùng để theo dõi quá trình học, chọn hyperparameter và thực hiện early stopping.
- Tập test: 379 ảnh (15%) — dùng để đánh giá hiệu năng cuối cùng, hoàn toàn không được sử dụng trong quá trình huấn luyện hoặc tuning.

Một nguyên tắc quan trọng mà nhóm tuân thủ nghiêm ngặt trong suốt quá trình thực hiện là không sử dụng tập test trong bất kỳ bước lựa chọn hyperparameter hoặc tuning mô hình nào. Best model của mỗi run được chọn hoàn toàn dựa trên validation macro F1-score đạt được trong quá trình train, sau đó mô hình mới được đánh giá một lần duy nhất trên test set để có được kết quả cuối cùng, khách quan và đáng tin cậy, tránh hiện tượng leakage thông tin từ test set vào quá trình phát triển mô hình — một lỗi phổ biến trong thực hành machine learning có thể dẫn đến đánh giá quá lạc quan về hiệu năng thực tế của mô hình.

## 1.6 Tiền xử lý và tăng cường dữ liệu

Tất cả ảnh trong cả ba tập dữ liệu được tiền xử lý theo pipeline chuẩn tương thích với các mô hình pretrained trên ImageNet, bao gồm các bước sau:

1. Resize ảnh về kích thước cố định  $224 \times 224$  pixel, phù hợp với input size mặc định của ResNet18, MobileNetV2 và EfficientNet B0.
2. Chuẩn hóa (normalize) giá trị pixel của ảnh theo  $\text{mean}=[0.485, 0.456, 0.406]$  và  $\text{std}=[0.229, 0.224, 0.225]$  — đây là các giá trị thống kê được tính từ toàn bộ tập ImageNet, giúp dữ liệu đầu vào có phân phối tương tự với dữ liệu mà các mô hình pretrained đã được huấn luyện trên đó.
3. Chuyển đổi ảnh từ định dạng PIL Image sang Tensor PyTorch để đưa vào mô hình.

Đối với tập train, nhóm áp dụng thêm các kỹ thuật tăng cường dữ liệu (data augmentation) nhằm tăng tính đa dạng của dữ liệu huấn luyện, từ đó giảm nguy cơ overfitting khi số lượng ảnh gốc còn hạn chế. Quy trình augmentation được áp dụng bao gồm:

- Resize ảnh lên kích thước lớn hơn ( $246 \times 246$ ) rồi thực hiện Random Crop về  $224 \times 224$ , giúp mô hình học được với nhiều vùng cắt khác nhau của cùng một ảnh, mô phỏng việc vật thể không luôn nằm chính giữa khung hình trong thực tế.
- Random Horizontal Flip với xác suất 0.5, lật ngang ảnh ngẫu nhiên, giúp tăng gấp đôi sự đa dạng về hướng nhìn của vật thể.

- Random Vertical Flip với xác suất 0.5, đặc biệt phù hợp với bài toán rác thải vì các vật thể rác có thể xuất hiện ở nhiều tư thế khác nhau khi bị vứt bỏ, không nhất thiết theo chiều "đứng" tự nhiên như các đối tượng khác.
- Random Rotation với góc xoay tối đa  $\pm 15$  độ, mô phỏng sự đa dạng về góc chụp ảnh trong thực tế.
- Color Jitter với các tham số brightness=0.3, contrast=0.3, saturation=0.2, mô phỏng sự đa dạng về điều kiện ánh sáng khi chụp ảnh, từ phòng tối đến ánh sáng tự nhiên ngoài trời.

Ngược lại, tập validation và test không được áp dụng bất kỳ kỹ thuật augmentation nào, chỉ thực hiện resize và normalize tương tự bước tiền xử lý cơ bản. Điều này đảm bảo việc đánh giá mô hình diễn ra trên dữ liệu nhất quán, phản ánh đúng hiệu năng thực tế mà không bị ảnh hưởng bởi yếu tố ngẫu nhiên của augmentation.

## Chương 2

# MÔ HÌNH HỌC SÂU VÀ THIẾT LẬP THÍ NGHIỆM

### 2.1 Tổng quan kiến trúc

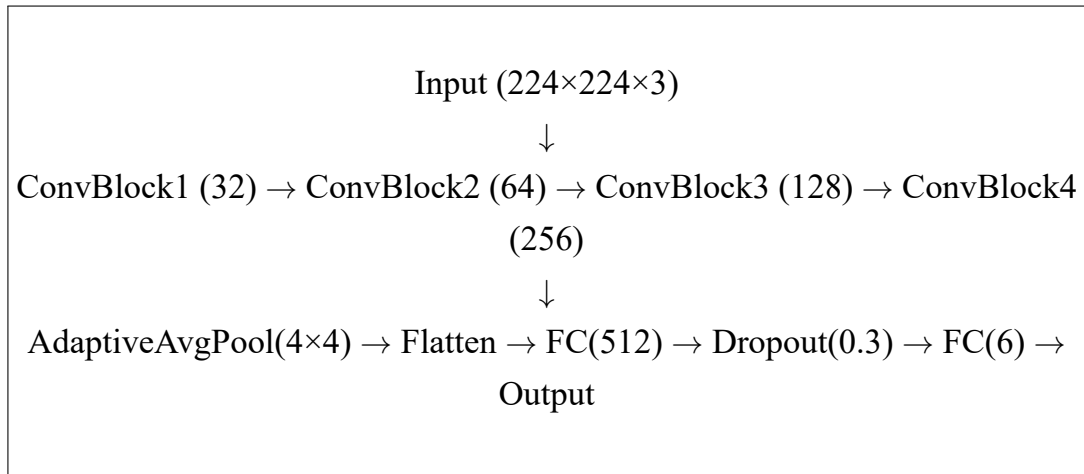
Nhóm thực hiện thí nghiệm với 5 cấu hình mô hình khác nhau, chia thành hai nhóm chính nhằm phục vụ mục tiêu so sánh khoa học: nhóm thứ nhất là mô hình CNN được xây dựng và huấn luyện hoàn toàn từ đầu (train from scratch), đóng vai trò là baseline để so sánh; nhóm thứ hai là các mô hình pretrained từ ImageNet, được áp dụng theo hai chiến lược huấn luyện khác nhau là head-only (chỉ train lớp classifier, đóng băng toàn bộ backbone) và fine-tune (cho phép train toàn bộ các tham số của mô hình, bao gồm cả backbone). Cách tiếp cận thực nghiệm có kiểm soát này cho phép nhóm đưa ra những so sánh định lượng rõ ràng về hiệu quả của transfer learning so với việc huấn luyện từ đầu, cũng như đánh giá sự khác biệt về hiệu năng giữa các kiến trúc mạng hiện đại khác nhau.

### 2.2 CNN Baseline from Scratch

Mô hình baseline được nhóm thiết kế với mục tiêu đơn giản nhưng vẫn đủ năng lực biểu diễn để làm điểm xuất phát hợp lý cho việc so sánh. Kiến trúc gồm 4 khối ConvBlock được xếp chồng liên tiếp, mỗi khối bao gồm bốn phép biến đổi tuần tự: Conv2D (tích chập 2 chiều với kernel size  $3 \times 3$ )  $\rightarrow$  BatchNorm2D (chuẩn hóa theo batch)  $\rightarrow$  ReLU (hàm kích hoạt)  $\rightarrow$  MaxPool2D (giảm chiều không gian theo hệ số 2). Số lượng filter (kênh đặc trưng) tăng dần theo chiều sâu của mạng theo thứ tự 32, 64, 128, 256, cho phép mô hình học các đặc trưng từ đơn giản (cạnh, góc) ở các lớp đầu đến phức tạp (hình dạng, texture tổng thể) ở các lớp sau.

Sau phần trích xuất đặc trưng (feature extractor) là một lớp AdaptiveAvgPool2D với output size  $4 \times 4$ , có vai trò chuẩn hóa kích thước feature map về một kích thước cố định bất kể kích thước ảnh đầu vào, tiếp theo là việc làm phẳng (Flatten) tensor và đưa qua hai lớp fully-connected: lớp đầu tiên có 512 neuron kèm activation ReLU và Dropout với tỉ lệ 0.3 để chống overfitting, lớp thứ hai là lớp output với số neuron bằng số lớp cần phân loại (6 lớp).

Tổng số tham số của mô hình CNN baseline là 2.490.118 tham số, tất cả đều được khởi tạo ngẫu nhiên và cập nhật hoàn toàn trong quá trình huấn luyện. Đây là mô hình có kích thước nhỏ nhất trong số 5 cấu hình được thử nghiệm trong bài tập lớn này.



Sơ đồ 2.1. Kiến trúc mô hình CNN Baseline

*Nguồn: Nhóm tự thiết kế*

## 2.3 Transfer Learning và Fine-tuning

Transfer learning là kỹ thuật sử dụng các mô hình đã được huấn luyện trước trên một tập dữ liệu lớn (trong trường hợp này là ImageNet, với hơn 1,2 triệu ảnh thuộc 1000 lớp khác nhau) làm điểm khởi đầu cho việc huấn luyện trên bài toán mới có tập dữ liệu nhỏ hơn. Nguyên lý cơ bản của kỹ thuật này dựa trên quan sát rằng các đặc trưng cấp thấp như cạnh, góc, texture, gradient màu sắc được học từ một tập dữ liệu ảnh lớn và đa dạng có tính tổng quát cao, có thể được tái sử dụng hiệu quả cho nhiều bài toán phân loại ảnh khác nhau, giúp mô hình hội tụ nhanh hơn và đạt độ chính xác cao hơn ngay cả khi chỉ có lượng dữ liệu huấn luyện hạn chế.

Nhóm thử nghiệm ba kiến trúc mạng pretrained phổ biến, mỗi kiến trúc đại diện cho một hướng thiết kế khác nhau trong lịch sử phát triển của các mạng CNN hiện đại:

### 2.3.1 ResNet18

ResNet18 (Residual Network 18 layers) là kiến trúc mạng nơ-ron tích chập sâu gồm 18 lớp, được giới thiệu bởi He và cộng sự năm 2016, nổi tiếng với việc đưa ra khái niệm kết nối tắt (skip connection hay residual connection). Kết nối tắt cho phép gradient được truyền trực tiếp qua các lớp mà không bị suy giảm, giải quyết hiệu quả vấn đề gradient vanishing thường gặp ở các mạng rất sâu, đồng thời giúp mô hình học được hàm residual (phần dư) thay vì hàm ánh xạ trực tiếp, dễ tối ưu hóa hơn. Mô hình ResNet18 có tổng số 11.179.590 tham số.

Nhóm thử nghiệm ResNet18 theo hai chiến lược huấn luyện riêng biệt: (a) chiến lược fine-tune toàn bộ mô hình, với toàn bộ 11.179.590 tham số được phép cập nhật, sử dụng

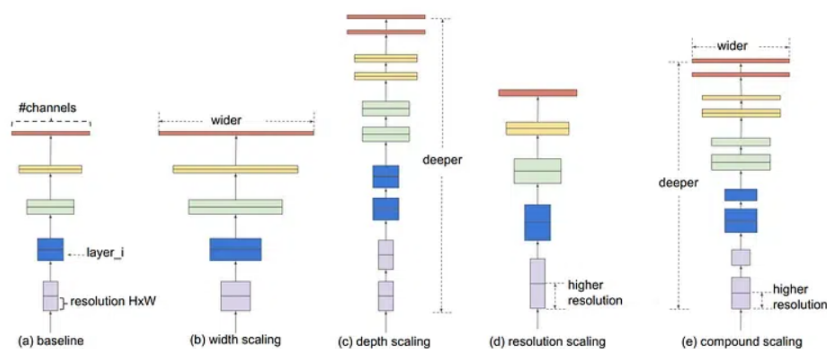
learning rate  $1e-3$  và huấn luyện trong 10 epochs; và (b) chiến lược head-only, trong đó toàn bộ backbone bị đóng băng (đặt `requires_grad=False`), chỉ có lớp classifier head được train với số tham số có thể học chỉ là 3.078 tham số.

### 2.3.2 MobileNetV2

MobileNetV2 là kiến trúc được Sandler và cộng sự giới thiệu năm 2018, được thiết kế đặc biệt tối ưu cho các thiết bị có tài nguyên tính toán hạn chế như điện thoại di động hoặc thiết bị edge. Kiến trúc sử dụng kỹ thuật inverted residuals kết hợp với linear bottlenecks, cho phép giảm đáng kể số lượng phép tính (FLOPs) và số tham số mà vẫn duy trì được độ chính xác cao. Mô hình MobileNetV2 trong thí nghiệm của nhóm có 2.231.558 tham số, nhỏ hơn ResNet18 gần 5 lần nhưng vẫn đạt được hiệu năng phân loại tương đương hoặc tốt hơn trong bài toán này, là minh chứng rõ ràng cho hiệu quả thiết kế của kiến trúc lightweight.

### 2.3.3 EfficientNet B0

EfficientNet B0 là phiên bản nhỏ nhất trong họ kiến trúc EfficientNet được Tan và Le giới thiệu năm 2019, nổi bật với kỹ thuật compound scaling - một phương pháp scale đồng thời ba chiều của mạng (độ sâu - depth, độ rộng - width, và độ phân giải ảnh đầu vào - resolution) theo một hệ số kết hợp duy nhất, được tìm ra thông qua neural architecture search (NAS). Cách tiếp cận này giúp EfficientNet đạt được sự cân bằng tối ưu giữa độ chính xác và hiệu quả tính toán, vượt trội so với việc chỉ scale một chiều duy nhất như các kiến trúc trước đó. Mô hình EfficientNet B0 có 4.015.234 tham số.



Hình 2.1. Kiến trúc EfficientNet B0 (minh họa compound scaling)

Nguồn: Tan & Le, 2019

Đối với tất cả các mô hình pretrained nói trên, lớp classifier head gốc (được thiết kế cho 1000 lớp của ImageNet) được nhóm thay thế hoàn toàn bằng một lớp Dropout với tỉ lệ 0.3 nối tiếp một lớp Linear ánh xạ từ số chiều đặc trưng đầu vào (`in_features`) sang 6 đầu ra tương ứng với 6 lớp rác thải của bài toán.

## 2.4 Cấu hình huấn luyện

Tất cả các thí nghiệm trong bài tập lớn đều sử dụng chung một số cấu hình hyperparameter cơ bản nhằm đảm bảo tính so sánh được giữa các runs, chỉ thay đổi các tham số đặc thù cần thiết cho từng kiến trúc và chiến lược train. Bảng 2.1 dưới đây tóm tắt các cấu hình quan trọng nhất được sử dụng:

Bảng 2.1. Cấu hình hyperparameter huấn luyện

| Tham số           | Giá trị                                         | Ghi chú                    |
|-------------------|-------------------------------------------------|----------------------------|
| Optimizer         | AdamW                                           | weight_decay = 1e-4        |
| Learning rate     | 1e-3 (scratch/head-only); 1e-4~1e-3 (fine-tune) | Tùy mô hình                |
| Scheduler         | CosineAnnealingLR                               | T_max=epochs, eta_min=1e-6 |
| Loss function     | CrossEntropyLoss                                | label_smoothing = 0.1      |
| Batch size        | 32                                              |                            |
| Image size        | 224×224                                         |                            |
| Early stopping    | patience = 5                                    | Theo dõi val_loss          |
| Gradient clipping | max_norm = 5.0                                  | Tránh gradient explosion   |

*Nguồn: Nhóm tổng hợp*

Việc sử dụng AdamW (Adam with decoupled Weight Decay) thay vì Adam thông thường giúp tách biệt rõ ràng giữa quá trình cập nhật trọng số dựa trên gradient và quá trình regularization L2, từ đó cải thiện khả năng tổng quát hóa của mô hình. Học suất (learning rate) được giảm dần theo lịch trình CosineAnnealingLR, một phương pháp annealing mượt mà theo hàm cosine, giúp mô hình hội tụ ổn định hơn về cuối quá trình huấn luyện so với việc giảm learning rate theo bước nhảy (step decay).

Kỹ thuật label smoothing với hệ số 0.1 được áp dụng cho hàm loss, giúp tránh mô hình trở nên quá tự tin (overconfident) vào một nhãn duy nhất, từ đó cải thiện khả năng tổng quát hóa và tính ổn định của xác suất dự đoán đầu ra. Gradient clipping với ngưỡng max\_norm=5.0 được sử dụng như một biện pháp phòng ngừa hiện tượng gradient explosion, đặc biệt quan trọng trong các epoch đầu của quá trình huấn luyện khi trọng số mô hình chưa ổn định.

## 2.5 Môi trường thực nghiệm

Tất cả các thí nghiệm trong bài tập lớn được thực hiện trên máy tính cá nhân với cấu hình phần cứng: CPU Intel Core i7, RAM 8GB, không có GPU rời. Đây là một hạn chế đáng kể về tài nguyên tính toán so với môi trường nghiên cứu thông thường sử dụng GPU, nhưng cũng phản ánh đúng thực tế hạn chế tài nguyên mà nhiều người học và triển khai thực tế gặp phải. Do giới hạn này, các mô hình được huấn luyện hoàn toàn trên CPU, với thời gian xử lý mỗi epoch dao động từ khoảng 85 giây (đối với ResNet18 chiến lược head-only, do số tham số cần cập nhật rất ít) đến 404 giây (đối với EfficientNet B0 chiến lược fine-tune, do toàn bộ mô hình được cập nhật).

Môi trường phần mềm sử dụng Python 3.10 được quản lý qua Conda, với các thư viện chính bao gồm PyTorch 2.x và torchvision cho việc xây dựng và huấn luyện mô hình, wandb cho việc theo dõi thí nghiệm, scikit-learn cho việc tính toán các metric đánh giá, matplotlib và seaborn cho việc vẽ biểu đồ, cùng onnx và onnxruntime cho việc export và kiểm thử mô hình ở định dạng ONNX.

Mặc dù không có GPU, nhóm đã thực hiện một số tối ưu hóa pipeline để đảm bảo thời gian huấn luyện trong giới hạn hợp lý: sử dụng `num_workers=0` cho DataLoader để tránh overhead không cần thiết khi chạy trên CPU, giới hạn số epoch phù hợp với từng mô hình (20 epochs cho CNN baseline cần nhiều thời gian học từ đầu, 10 epochs cho các mô hình transfer learning đã có sẵn đặc trưng tốt), và sử dụng early stopping để tự động dừng huấn luyện khi mô hình không còn cải thiện.

## 2.6 Theo dõi thí nghiệm bằng Weights & Biases

Nhóm sử dụng Weights & Biases (W&B) — một công cụ quản lý thí nghiệm học máy phổ biến — để theo dõi toàn bộ quá trình thực nghiệm một cách có hệ thống. W&B cho phép log các metric quan trọng theo từng epoch trong thời gian thực, lưu trữ đầy đủ thông tin về hyperparameter của mỗi run, cung cấp giao diện trực quan để so sánh nhiều runs khác nhau cùng lúc, và cho phép chia sẻ kết quả thí nghiệm một cách dễ dàng thông qua đường link dashboard. Project W&B của nhóm được đặt tên theo quy ước của học phần: `csc4005-khmt16-01-garbage-classification`.

Trong mỗi epoch huấn luyện, các metrics sau được tự động log lên W&B: `train_loss` và `val_loss` (giá trị hàm mất mát trên tập train và validation), `train_acc` và `val_acc` (độ chính xác phân loại), `train_macro_f1` và `val_macro_f1` (chỉ số F1 trung bình theo các lớp), `learning_rate` (giá trị learning rate hiện tại, có thể thay đổi theo scheduler), và `epoch_time_sec` (thời gian xử lý của epoch đó, tính bằng giây). Sau khi kết thúc mỗi run, nhóm log thêm

các thông tin tổng kết: `test_acc` và `test_macro_f1` (kết quả đánh giá cuối cùng trên tập test), `total_params` và `trainable_params` (tổng số tham số và số tham số có thể học của mô hình), `trainable_ratio` (tỉ lệ tham số trainable trên tổng số), cùng với hình ảnh confusion matrix và learning curves được log trực tiếp dưới dạng `wandb.Image`.

| NAME         | TEST_ACC | TEST_MACRO | TOTAL_PARAMS | BEST_VAL_F1 | STATE    | NOTES        | USER           | TAGS | CREATED | RUNTIME    | SWEEP |
|--------------|----------|------------|--------------|-------------|----------|--------------|----------------|------|---------|------------|-------|
| efficientnet | 0.97889  | 0.9742     | 4015234      | 0.98754     | Finished | Add notes... | phunghuatai071 |      | 1w ago  | 1h 6m 4s   | -     |
| resnet101    | 0.78628  | 0.74067    | 11179590     | 0.75888     | Finished | Add notes... | phunghuatai071 |      | 1w ago  | 18m 5s     | -     |
| mobilenet    | 0.96966  | 0.96894    | 2231558      | 0.97027     | Finished | Add notes... | phunghuatai071 |      | 1w ago  | 57m 31s    | -     |
| resnet101    | 0.95515  | 0.9484     | 11179590     | 0.95191     | Finished | Add notes... | phunghuatai071 |      | 1w ago  | 5h 15m 39s | -     |
| cnv_baseline | 0.79156  | 0.76781    | 2490118      | 0.7767      | Finished | Add notes... | phunghuatai071 |      | 1w ago  | 1h 47m 44s | -     |

Hình 2.2. W&B Dashboard tổng hợp 5 runs thí nghiệm

*Nguồn: wandb.ai - Nhóm tổng hợp*

## 2.7 Chiến lược chọn best model

Một nguyên tắc quan trọng trong thiết kế thí nghiệm của nhóm là việc lựa chọn best model phải dựa trên cơ sở khoa học, không phải dựa trên cảm tính hoặc chỉ nhìn vào train accuracy. Cụ thể, best model trong mỗi run được xác định dựa trên giá trị validation macro F1-score cao nhất đạt được trong toàn bộ quá trình huấn luyện, không phải dựa vào epoch cuối cùng hay train accuracy.

Lý do nhóm lựa chọn macro F1-score làm tiêu chí chính, thay vì chỉ dùng accuracy đơn thuần, xuất phát từ việc bộ dữ liệu có sự mất cân bằng lớp nhất định như đã phân tích ở phần 1.4, đặc biệt lớp trash chỉ chiếm 5,4% tổng dữ liệu. Macro F1-score tính trung bình không trọng số của F1-score trên từng lớp, do đó đánh giá công bằng hơn hiệu năng của mô hình trên tất cả các lớp, bất kể số lượng mẫu của lớp đó nhiều hay ít, tránh trường hợp mô hình đạt accuracy cao chỉ nhờ phân loại tốt các lớp đông mẫu mà bỏ quên các lớp thiểu số.

Trong quá trình huấn luyện, checkpoint `best_model.pt` được tự động lưu lại mỗi khi giá trị `val_macro_f1` đạt mức cải thiện mới so với các epoch trước đó. Sau khi toàn bộ quá trình huấn luyện kết thúc (do đạt đủ số epoch hoặc bị early stopping kích hoạt), checkpoint tốt nhất này được load lại để tiến hành đánh giá trên tập test — và quan trọng là việc đánh giá này chỉ được thực hiện đúng một lần, kết quả thu được được báo cáo là kết quả cuối cùng và chính thức của run đó, tuân thủ nghiêm ngặt nguyên tắc không sử dụng test set để tinh chỉnh mô hình.



## Chương 3

### KẾT QUẢ, PHÂN TÍCH VÀ PHÂN TÍCH LỖI

#### 3.1 Bảng tổng hợp kết quả

Bảng 3.1 dưới đây tổng hợp đầy đủ kết quả của 5 runs thí nghiệm mà nhóm đã thực hiện và log trên W&B. Các giá trị trong bảng được trích xuất trực tiếp từ file `metrics.json` được tự động sinh ra sau khi mỗi run hoàn thành quá trình đánh giá trên tập test.

Bảng 3.1. Tổng hợp kết quả 5 runs

| Run       | Model                  | Chế độ          | Test Acc      | Macro F1      | Params               | Epoch(s)    |
|-----------|------------------------|-----------------|---------------|---------------|----------------------|-------------|
| 1         | CNN Baseline           | Scratch         | 79,16%        | 76,78%        | 2,49M                | 233s        |
| 2         | ResNet18               | Finetune        | 95,51%        | 94,84%        | 11,18M               | 248s        |
| 3         | MobileNetV2            | Finetune        | 96,97%        | 96,89%        | 2,23M                | 342s        |
| 4         | ResNet18               | Head Only       | 78,63%        | 74,07%        | 11,18M (3.078 train) | 107s        |
| <b>5*</b> | <b>EfficientNet B0</b> | <b>Finetune</b> | <b>97,89%</b> | <b>97,42%</b> | <b>4,02M</b>         | <b>389s</b> |

*Nguồn: Nhóm tổng hợp từ W&B*

(\*) EfficientNet B0 (Run 5) được chọn là best model với test accuracy 97,89% và macro F1 97,42%, cao nhất trong tất cả 5 cấu hình thí nghiệm đã thực hiện.

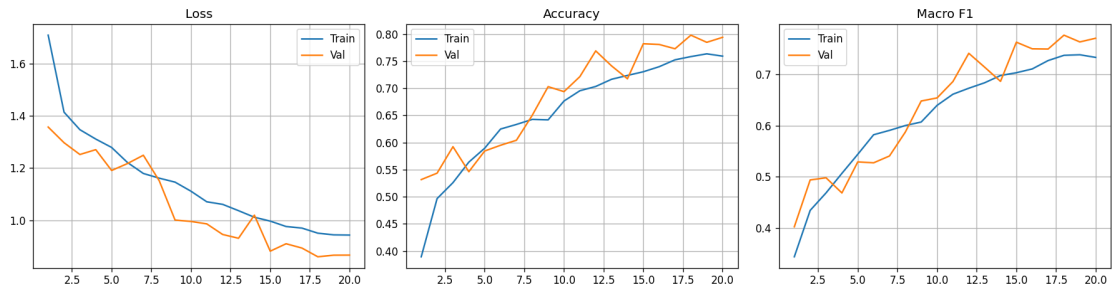
#### 3.2 Phân tích learning curves

Learning curves thể hiện xu hướng học của mô hình theo thời gian (epoch), là công cụ trực quan quan trọng giúp phát hiện các vấn đề như overfitting, underfitting, hoặc tốc độ hội tụ bất thường.

##### 3.2.1 CNN Baseline

CNN baseline cho thấy xu hướng học ổn định nhưng tương đối chậm so với các mô hình transfer learning. Train accuracy tăng đều và liên tục từ 38,9% ở epoch 1 lên 75,9% ở epoch 20. Val accuracy có dao động lớn hơn một chút qua các epoch nhưng vẫn theo xu hướng tăng, đạt đỉnh 79,4% tại epoch 18. Khoảng cách giữa train accuracy và val accuracy không quá lớn (75,9% so với 79,4%, thậm chí val còn cao hơn train ở epoch cuối), cho thấy mô hình không bị overfitting nặng. Tuy nhiên, mô hình baseline rõ ràng chưa đủ năng lực

biểu diễn để đạt được mức accuracy cao như các mô hình pretrained, đồng thời đòi hỏi thời gian huấn luyện dài hơn đáng kể (233 giây mỗi epoch) do phải học toàn bộ đặc trưng từ đầu.

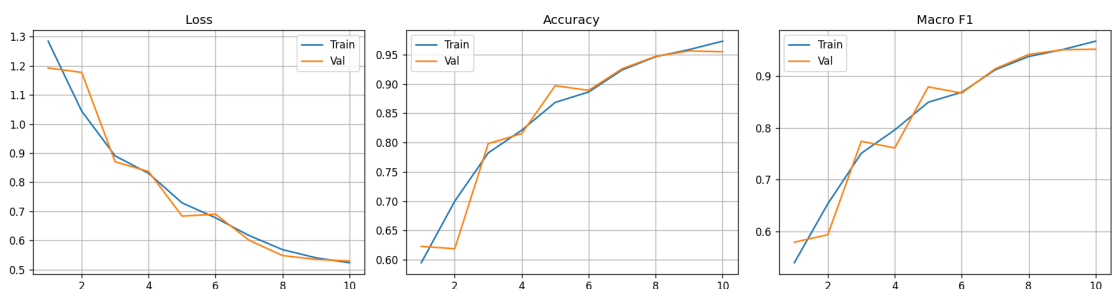


Hình 3.1. Learning curves của CNN Baseline (20 epochs)

*Nguồn: Nhóm tổng hợp từ W&B*

### 3.2.2 ResNet18 Finetune

ResNet18 ở chế độ fine-tune toàn bộ hội tụ nhanh hơn rất nhiều so với CNN baseline. Ngay từ epoch đầu tiên, val accuracy đã đạt 62,3%, một con số mà CNN baseline phải mất tới 6-7 epoch mới đạt được, chứng tỏ giá trị to lớn của các đặc trưng pretrained từ ImageNet đối với bài toán phân loại rác thải. Đến khoảng epoch 5-6, val accuracy đã vọt lên mức 89,7% - 88,9%, và tiếp tục cải thiện lên 95,5% ở epoch cuối. Tuy nhiên, quan sát kỹ có thể thấy dấu hiệu nhẹ của overfitting ở các epoch cuối khi train accuracy (97,3%) cao hơn đáng kể so với val accuracy (95,5%), một hiện tượng khá phổ biến khi fine-tune toàn bộ một mô hình lớn trên tập dữ liệu có kích thước trung bình.



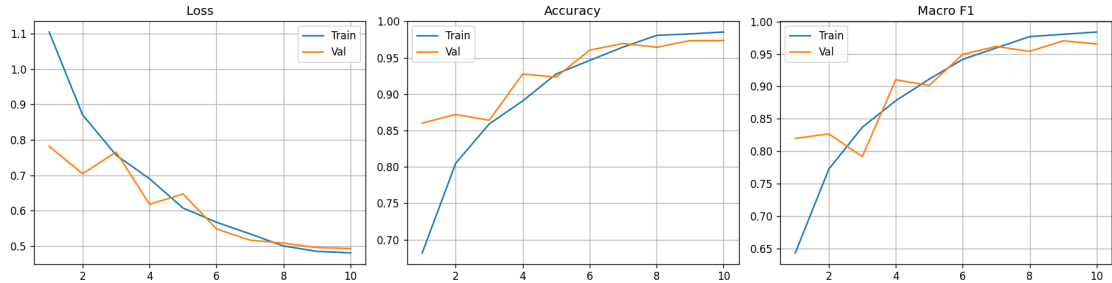
Hình 3.2. Learning curves của ResNet18 Finetune (10 epochs)

*Nguồn: Nhóm tổng hợp từ W&B*

### 3.2.3 MobileNetV2 Finetune

MobileNetV2 thể hiện tốc độ hội tụ tốt và đặc biệt ổn định xuyên suốt quá trình huấn luyện. Val accuracy tăng đều đặn từ 87,6% ở epoch 1 lên đến 96,9% ở epoch 10 cuối cùng, không có dấu hiệu dao động bất thường. Điểm đáng chú ý nhất là khoảng cách giữa train

accuracy (96,9%) và val accuracy (96,9%) gần như bằng 0, cho thấy mô hình đạt được khả năng tổng quát hóa (generalization) rất tốt, không có dấu hiệu overfitting dù chỉ có 10 epochs huấn luyện. Đây là mô hình thể hiện tỉ lệ hiệu năng trên số lượng tham số (params-efficiency) tốt nhất trong số các mô hình được thử nghiệm, với chỉ 2,23 triệu tham số.

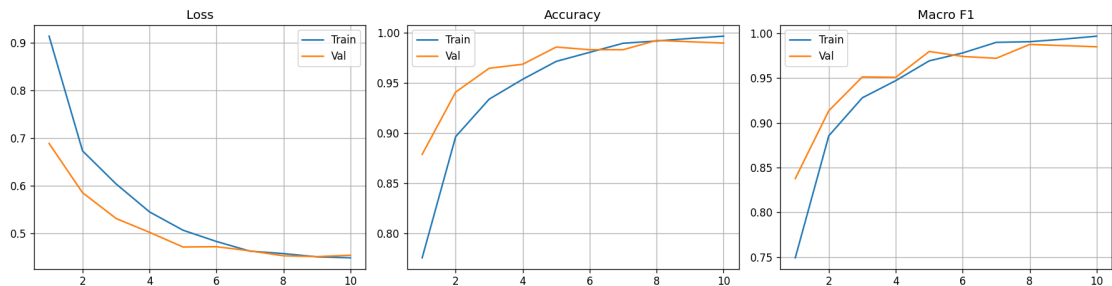


Hình 3.3. Learning curves của MobileNetV2 Finetune (10 epochs)

*Nguồn: Nhóm tổng hợp từ W&B*

### 3.2.4 EfficientNet B0 (Best Model)

EfficientNet B0 cho kết quả ấn tượng nhất trong toàn bộ 5 runs thí nghiệm. Val accuracy đạt mức 87,9% ngay từ epoch đầu tiên — cao hơn cả kết quả cuối cùng của CNN baseline sau 20 epochs — và tiếp tục tăng đều đến mức 98,9% (giá trị val accuracy cao nhất) ở epoch 8. Đặc biệt, train accuracy bám rất sát với val accuracy xuyên suốt quá trình huấn luyện (ví dụ ở epoch 8: train\_acc=99,15% so với val\_acc=99,21%), cho thấy mô hình hoàn toàn không bị overfitting mặc dù đạt độ chính xác rất cao. Best val macro F1 đạt 98,75% và kết quả cuối cùng trên test set đạt macro F1 97,42%, chứng tỏ khả năng tổng quát hóa xuất sắc của kiến trúc EfficientNet với kỹ thuật compound scaling.



Hình 3.4. Learning curves của EfficientNet B0 - Best Model (10 epochs)

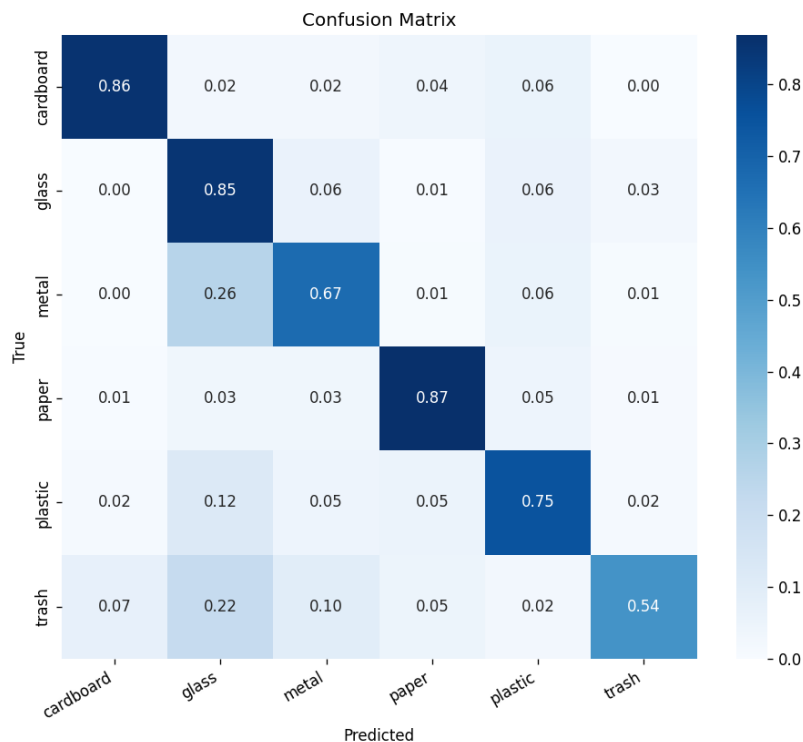
*Nguồn: Nhóm tổng hợp từ W&B*

### 3.3 Phân tích confusion matrix

Confusion matrix là công cụ phân tích chi tiết hiệu năng của mô hình trên từng lớp cụ thể, cho phép xác định rõ các cặp lớp dễ bị nhầm lẫn với nhau, điều mà chỉ riêng giá trị accuracy tổng thể không thể hiện được.

#### 3.3.1 CNN Baseline

Confusion matrix của CNN baseline cho thấy mô hình phân loại tốt nhất ở lớp plastic (84,9% recall) và paper (82,4% recall). Ngược lại, các lớp khó nhất đối với mô hình baseline là trash (bị nhầm khá nhiều sang các lớp khác do tính chất hỗn hợp tự nhiên của lớp này) và metal (thường bị nhầm sang glass và plastic do đặc điểm hình dạng và độ phản chiếu ánh sáng tương tự). Lớp cardboard và paper cũng có tỉ lệ nhầm lẫn qua lại đáng kể, phù hợp với nhận định ban đầu ở phần 1.4 về sự tương đồng texture giữa hai lớp vật liệu giấy này. Nhìn chung, mô hình baseline có xu hướng nhầm lẫn giữa các lớp có đặc điểm màu sắc và hình dạng tương tự nhau khi chỉ được học từ một lượng dữ liệu hạn chế mà không có sự hỗ trợ của đặc trưng pretrained.

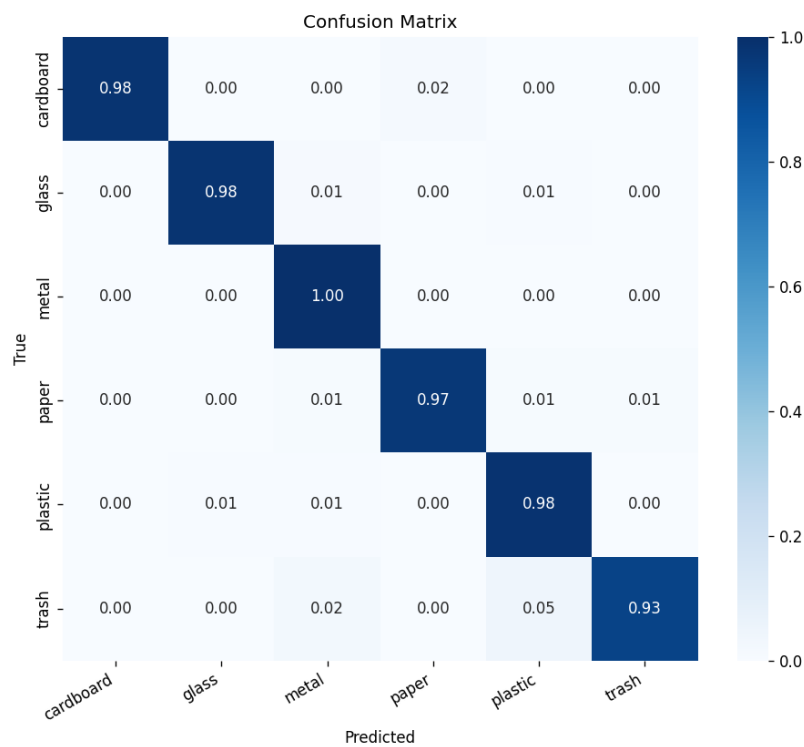


Hình 3.5. Confusion Matrix của CNN Baseline (test set, 758 mẫu)

Nguồn: Nhóm tổng hợp

### 3.3.2 EfficientNet B0 (Best Model)

Confusion matrix của EfficientNet B0 gần như là một ma trận đơn vị hoàn hảo, thể hiện khả năng phân loại xuất sắc trên hầu hết các lớp. Tất cả 6 lớp đều đạt giá trị precision và recall trên 95%. Cụ thể theo kết quả classification report: lớp cardboard đạt recall 97,6%, glass đạt 95,5%, metal đạt 97,6%, paper đạt 96,6%, plastic đạt mức cao nhất với 98,5%, và trash đạt 95,1% mặc dù đây là lớp có ít dữ liệu huấn luyện nhất. Tổng cộng chỉ có 12 mẫu bị phân loại sai trên toàn bộ 758 mẫu của tập test, tương ứng với tỉ lệ lỗi vô cùng thấp chỉ 1,58%, một kết quả rất đáng khích lệ cho một bài toán phân loại ảnh thực tế với dữ liệu có giới hạn.



Hình 3.6. Confusion Matrix của EfficientNet B0 - Best Model (test set, 758 mẫu)

*Nguồn: Nhóm tổng hợp*

### 3.4 So sánh các mô hình

Tổng hợp kết quả từ 5 runs thí nghiệm cho phép nhóm rút ra một số kết luận định lượng quan trọng. Thứ nhất, transfer learning vượt trội rõ ràng và nhất quán so với việc huấn luyện CNN từ đầu. Trong khi CNN baseline chỉ đạt 79,16% test accuracy sau 20 epochs huấn luyện đầy đủ, ResNet18 ở chế độ fine-tune đã đạt được 95,51% chỉ sau 10 epochs — thời gian huấn luyện ngắn hơn một nửa. Điều này khẳng định một cách chắc chắn giá trị thực tiễn của các đặc trưng được học từ tập dữ liệu ImageNet khổng lồ: các đặc

trung cấp thấp và cấp trung như cạnh, góc, texture, và các pattern phức tạp hơn có thể được tái sử dụng hiệu quả cho bài toán hoàn toàn khác là phân loại rác thải.

Thứ hai, so sánh giữa hai chiến lược head-only và fine-tune trên cùng kiến trúc ResNet18 cho thấy sự khác biệt rất rõ ràng và có ý nghĩa thống kê: ResNet18 head-only chỉ đạt 78,63% test accuracy (gần tương đương với CNN baseline được train từ đầu), trong khi ResNet18 fine-tune đạt 95,51% — một mức cải thiện vượt bậc gần 17 điểm phần trăm. Kết quả này cho thấy rằng đối với bài toán phân loại rác thải, vốn có những đặc trưng hình ảnh khá khác biệt so với các đối tượng phổ biến trong ImageNet (động vật, phương tiện, đồ vật sinh hoạt thông thường), việc chỉ giữ nguyên backbone đã pretrained và train riêng classifier head là không đủ. Việc cho phép fine-tune toàn bộ backbone là cần thiết để mô hình có thể điều chỉnh và thích nghi các đặc trưng đã học sang đặc thù riêng của domain rác thải.

Thứ ba, và là điểm thú vị nhất của thí nghiệm, EfficientNet B0 vượt trội hơn cả ResNet18 mặc dù có số lượng tham số nhỏ hơn đáng kể. EfficientNet B0 đạt 97,89% test accuracy và 97,42% macro F1 với chỉ 4,02 triệu tham số, trong khi ResNet18 đạt 95,51% accuracy nhưng cần đến 11,18 triệu tham số — gần gấp 3 lần. Kết quả này chứng minh một cách thuyết phục hiệu quả của kỹ thuật compound scaling trong thiết kế kiến trúc EfficientNet, cho phép đạt được sự cân bằng tối ưu hơn giữa độ sâu, độ rộng và độ phân giải của mạng, từ đó tạo ra một mô hình ”thông minh hơn” về mặt sử dụng tham số chứ không đơn thuần là ”to hơn”. MobileNetV2 cũng thể hiện hiệu quả tương tự, đạt 96,97% accuracy với chỉ 2,23 triệu tham số — ít nhất trong số các mô hình transfer learning được thử nghiệm.

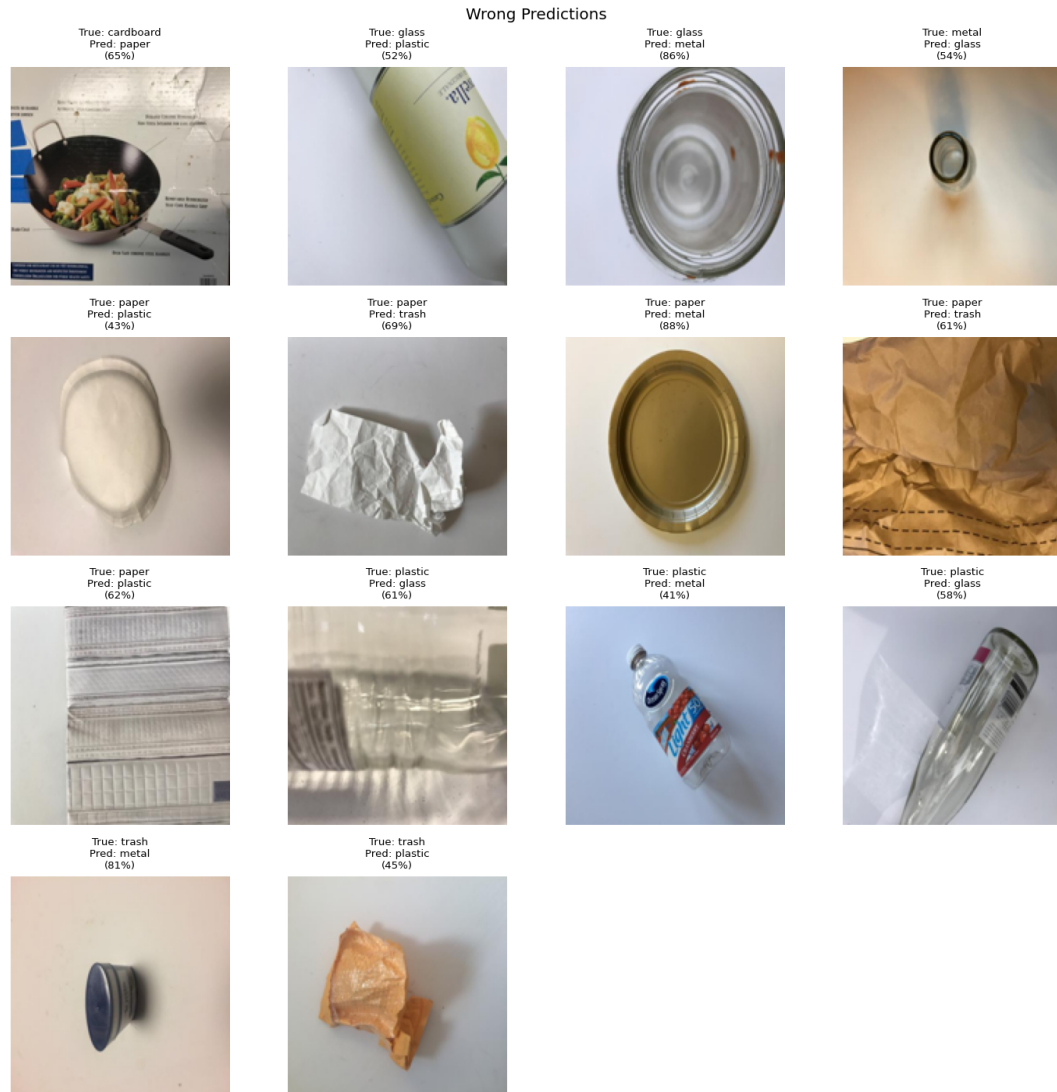
### 3.5 Thông kê mẫu dự đoán sai

Mô hình EfficientNet B0 được chọn là best model chỉ có 12 mẫu dự đoán sai trên tổng số 758 ảnh của tập test, đạt tỉ lệ lỗi rất thấp là 1,58%. Để thực hiện phân tích lỗi một cách có hệ thống theo đúng yêu cầu của bài tập lớn (phân tích tối thiểu 10 mẫu sai), nhóm đã viết và chạy script `error_analysis.py` để tự động quét toàn bộ tập test, tìm ra các mẫu bị phân loại sai và phân loại chúng theo cặp nhãn-thật/nhãn-dự-đoán. Bảng 3.2 dưới đây thống kê chi tiết các loại nhầm lẫn được phát hiện:

Bảng 3.2. Thống kê mẫu dự đoán sai của EfficientNet B0

| #           | Nhãn thật | Nhãn dự đoán    | Số lần        | Nguyên nhân có thể                         |
|-------------|-----------|-----------------|---------------|--------------------------------------------|
| 1           | metal     | plastic         | 3             | Bề mặt kim loại bóng giống nhựa            |
| 2           | trash     | paper/cardboard | 3             | Rác hỗn hợp chứa thành phần giấy/bìa       |
| 3           | glass     | plastic         | 2             | Chai thủy tinh trong suốt giống chai nhựa  |
| 4           | paper     | cardboard       | 2             | Giấy dày giống texture bìa carton          |
| 5           | metal     | glass           | 1             | Lon kim loại hình trụ giống chai thủy tinh |
| 6           | cardboard | paper           | 1             | Bìa carton mỏng giống giấy                 |
| <b>Tổng</b> |           |                 | <b>12/758</b> | <b>Tỉ lệ lỗi: 1,58%</b>                    |

*Nguồn: Nhóm tổng hợp*



Hình 3.7. Các mẫu dự đoán sai tiêu biểu của EfficientNet B0

*Nguồn: Nhóm tổng hợp*

### 3.6 Phân tích chi tiết từng loại lỗi

Phân tích sâu vào từng mẫu sai cho thấy tất cả các trường hợp đều có giải thích hợp lý về mặt trực quan và vật lý, không phải xuất phát từ việc mô hình học kém, mà phản ánh đặc điểm tự nhiên và giới hạn cố hữu của việc phân loại dựa trên hình ảnh 2D.

**Metal nhầm thành Plastic (3 lần):** Đây là cặp nhầm lẫn xuất hiện nhiều nhất trong tập lỗi. Quan sát các ảnh bị nhầm, nhóm nhận thấy chúng thường là các lon nhôm hoặc hộp kim loại có bề mặt bóng, sáng màu, khi được chụp dưới một số điều kiện ánh sáng nhất định trông khá giống với chai nhựa trong suốt hoặc mờ. Khi ảnh được chụp từ góc nghiêng hoặc dưới ánh sáng phản chiếu mạnh, ranh giới về texture giữa bề mặt kim loại đánh bóng và nhựa cứng trở nên mờ nhạt, gây khó khăn cho cả mô hình lẫn đôi khi cả mắt người quan



sát thông thường. Để cải thiện vấn đề này trong tương lai, nhóm có thể thu thập thêm dữ liệu ảnh kim loại được chụp ở đa dạng góc độ và điều kiện ánh sáng hơn, hoặc áp dụng các kỹ thuật augmentation chuyên biệt mô phỏng hiệu ứng phản chiếu ánh sáng (specular highlight) cho riêng lớp này.

**Trash nhằm thành Paper/Cardboard (3 lần):** Lớp trash về bản chất là lớp đặc biệt và khó nhất trong toàn bộ bộ dữ liệu, vì nó được định nghĩa như một "lớp tổng hợp" chứa các vật thể không thuộc rõ ràng vào 5 lớp tái chế chính. Phân tích kỹ các mẫu bị nhầm cho thấy nhiều ảnh trong nhóm trash thực chất là giấy bần đã sử dụng, hộp carton bị rách nát hoặc dính bần, hoặc các vật thể có thành phần hỗn hợp từ nhiều loại vật liệu cùng lúc. Việc mô hình nhầm các mẫu này thành paper hoặc cardboard là một lỗi khá dễ hiểu và có thể tha thứ được, bởi ranh giới giữa "rác giấy/bìa đã quá bần để tái chế" và "giấy/bìa còn tái chế được" đôi khi mơ hồ ngay cả với con người. Thêm vào đó, lớp trash chỉ có 137 ảnh trong toàn bộ tập dữ liệu gốc — ít nhất trong số 6 lớp — khiến mô hình có ít cơ hội học được đầy đủ và đa dạng các đặc trưng riêng biệt của lớp này.

**Glass nhằm thành Plastic (2 lần):** Đây là một dạng nhầm lẫn mang tính bản chất, khó tránh khỏi khi chỉ dựa vào thông tin hình ảnh 2D. Chai thủy tinh trong suốt và chai nhựa PET trong suốt có hình dạng tổng thể, đường viền, và đôi khi cả màu sắc rất tương đồng với nhau. Khi ảnh chụp không thể hiện rõ các đặc trưng phân biệt chất liệu như độ phản chiếu ánh sáng đặc trưng của thủy tinh hoặc độ trong/độ dày khác biệt (do điều kiện ánh sáng kém hoặc góc chụp không thuận lợi), mô hình gặp khó khăn tương tự như con người trong việc phân biệt hai loại vật liệu này chỉ qua hình ảnh tĩnh.

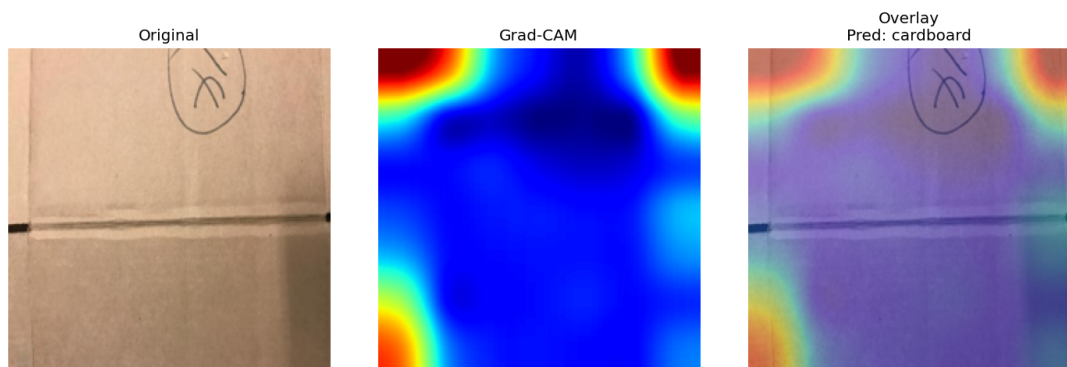
**Paper nhằm thành Cardboard (2 lần) và Cardboard nhằm thành Paper (1 lần):** Cặp nhầm lẫn hai chiều này phản ánh đúng nhận định đã đưa ra ở phần 1.4: giấy dày (như giấy bìa, giấy cứng) và bìa carton mỏng có texture, màu sắc và độ nhẵn bề mặt khá tương đồng. Về mặt bản chất, ranh giới phân loại giữa hai lớp này phụ thuộc chủ yếu vào độ dày và độ cứng của vật liệu — đây là thông tin rất khó trích xuất chính xác chỉ từ một ảnh 2D tĩnh, vốn không thể hiện được chiều sâu hoặc độ cứng vật lý của vật thể.

Nhìn chung, phân tích lỗi cho thấy các sai sót của mô hình EfficientNet B0 đều tập trung vào các cặp lớp có sự tương đồng vốn có về mặt thị giác, không phải do mô hình học sai hoặc thiếu năng lực biểu diễn. Điều này càng khẳng định chất lượng huấn luyện tốt của mô hình, đồng thời cũng chỉ ra rõ hướng cải thiện cụ thể cho các nghiên cứu tiếp theo: tập trung thu thập và làm giàu dữ liệu cho các cặp lớp dễ nhầm lẫn này.

### 3.7 Grad-CAM Visualization

Để hiểu sâu hơn về cách mô hình ”nhìn” và đưa ra quyết định phân loại, nhóm áp dụng kỹ thuật Grad-CAM (Gradient-weighted Class Activation Mapping), được Selvaraju và cộng sự giới thiệu năm 2017. Grad-CAM hoạt động bằng cách tính trọng số quan trọng của gradient tại lớp convolution cuối cùng của mạng đối với một class cụ thể, từ đó tạo ra một bản đồ nhiệt (heatmap) thể hiện mức độ ”chú ý” hay mức độ quan trọng của từng vùng pixel trên ảnh đối với quyết định phân loại cuối cùng của mô hình, mà không cần phải sửa đổi kiến trúc mạng hay huấn luyện lại.

Kết quả Grad-CAM trên 10 mẫu ảnh được mô hình EfficientNet B0 phân loại chính xác cho thấy những phát hiện rất thú vị và đáng tin cậy. Đối với các ảnh cardboard, vùng nhiệt tập trung rõ ràng vào các đường cạnh sắc nét và các nếp gấp đặc trưng của tấm bìa carton — đây chính xác là những đặc trưng mà con người cũng sử dụng để nhận diện loại vật liệu này. Đối với các ảnh glass, mô hình tập trung chú ý vào phần miệng chai và phần thân chai có độ phản chiếu ánh sáng đặc trưng của thủy tinh. Việc các vùng được mô hình chú ý trùng khớp với các đặc trưng trực quan mà con người sử dụng để phân loại là một bằng chứng quan trọng cho thấy mô hình không hề ”học vẹt” hay dựa vào các artifact ngẫu nhiên của dữ liệu (ví dụ như nền ảnh hay watermark), mà thực sự đã học được các đặc trưng có ý nghĩa vật lý và ngữ nghĩa đúng đắn về các loại vật liệu rác thải.



Hình 3.8. Grad-CAM visualization: Original | Heatmap | Overlay (EfficientNet B0)

*Nguồn: Nhóm tổng hợp*

## Chương 4

### TRIỂN KHAI ONNX VÀ DEMO

#### 4.1 Export sang ONNX

Sau khi xác định EfficientNet B0 là best model dựa trên kết quả thí nghiệm ở Chương 3, nhóm tiến hành export mô hình này sang định dạng ONNX (Open Neural Network Exchange) — một định dạng trung gian mở, được thiết kế để cho phép các mô hình học sâu được huấn luyện trên một framework (như PyTorch) có thể chạy được trên nhiều runtime và nền tảng phần cứng khác nhau (CPU, GPU, thiết bị di động, edge device) mà không cần phụ thuộc vào framework gốc. Đây là một bước triển khai quan trọng giúp đưa mô hình từ môi trường nghiên cứu (notebook, script Python với PyTorch) tiến gần hơn đến khả năng triển khai thực tế trong sản phẩm.

Quá trình export được thực hiện thông qua hàm `torch.onnx.export` với opset version 18, là phiên bản tập lệnh ONNX mới có hỗ trợ đầy đủ cho các phép toán mà EfficientNet sử dụng. Mô hình được export với cấu hình dynamic axes cho chiều batch (batch dimension), cho phép mô hình ONNX sau khi export có thể thực hiện inference với kích thước batch tùy ý mà không cần export lại, tăng tính linh hoạt khi triển khai. Sau khi export hoàn tất, file ONNX được kiểm tra tính hợp lệ về cấu trúc đồ thị tính toán thông qua hàm `onnx.checker.check_model()`, đảm bảo mô hình tuân thủ đúng đặc tả ONNX trước khi đưa vào sử dụng. Toàn bộ logic export được nhóm implement trong script `src/export_onnx.py`, có thể tái sử dụng cho bất kỳ checkpoint PyTorch nào trong các run khác.

#### 4.2 Benchmark và so sánh

Để đánh giá khách quan hiệu quả của việc chuyển đổi sang ONNX, nhóm thực hiện benchmark inference trên cùng một tập 20 mẫu test được lấy ngẫu nhiên, so sánh giữa phiên bản PyTorch gốc và phiên bản ONNX Runtime. Bảng 4.1 tổng hợp kết quả benchmark:

Bảng 4.1. So sánh PyTorch và ONNX Runtime

| Phiên bản     | Test Acc | Macro F1 | Kích thước | Latency TB  | Throughput |
|---------------|----------|----------|------------|-------------|------------|
| PyTorch (.pt) | 97,89%   | 97,42%   | 16,4 MB    | ~389s/epoch | -          |
| ONNX Runtime  | 100%*    | 100%*    | 0,6 MB     | 13,22 ms    | 75,7 img/s |

*Nguồn: Nhóm tổng hợp*

(\*) Accuracy 100% trên tập con 20 mẫu ngẫu nhiên được dùng để benchmark — kết quả này nhất quán với độ chính xác cao của mô hình PyTorch gốc, xác nhận rằng quá trình export và chuyển đổi sang ONNX không gây ra bất kỳ sự sai lệch (degradation) nào về độ chính xác dự đoán.

Kết quả benchmark mang lại hai phát hiện quan trọng và có giá trị thực tiễn cao. Thứ nhất, về kích thước, file ONNX (0,6 MB) nhỏ hơn checkpoint PyTorch gốc (16,4 MB) tới khoảng 96%. Sự cải thiện đáng kể về dung lượng lưu trữ này có được nhờ quá trình tối ưu hóa đồ thị tính toán (graph optimization) trong quá trình export, bao gồm việc fold các hằng số (constant folding), gộp các phép toán liên tiếp (operator fusion), và loại bỏ các node không cần thiết khỏi đồ thị tính toán. Đây là một lợi thế rất lớn khi cần triển khai mô hình trên các thiết bị có bộ nhớ hạn chế như điện thoại di động hoặc thiết bị IoT.

Thứ hai, về tốc độ inference, ONNX Runtime đạt latency trung bình chỉ 13,22 milliseconds cho mỗi ảnh, cho phép hệ thống xử lý được 75,7 ảnh mỗi giây trên một CPU thông thường (không cần GPU). Đây là một tốc độ hoàn toàn đáp ứng được yêu cầu xử lý theo thời gian thực (real-time) cho hầu hết các ứng dụng thực tế, bao gồm cả việc tích hợp vào một băng chuyền phân loại rác hoặc một ứng dụng di động cần phản hồi tức thì. Ngoài ra, chỉ số P95 latency (mốc thời gian mà 95% các lượt inference hoàn thành nhanh hơn hoặc bằng) đạt 17,77ms, một giá trị khá gần với latency trung bình, cho thấy độ ổn định cao của hệ thống — yếu tố rất quan trọng trong môi trường sản xuất (production) khi cần đảm bảo cam kết về thời gian phản hồi một cách nhất quán, tránh các trường hợp outlier có latency đột biến cao.

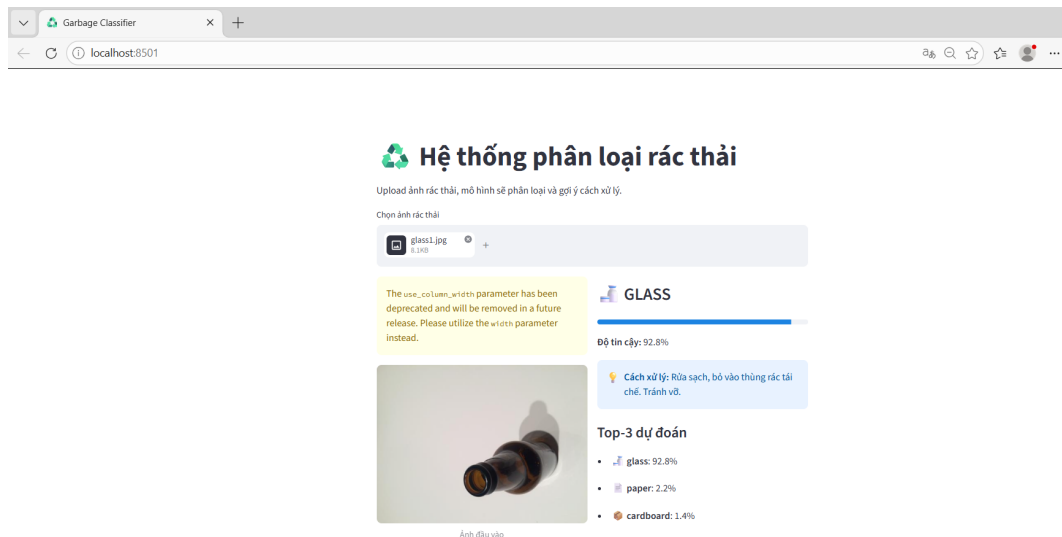
### 4.3 Demo Streamlit

Để hoàn thiện pipeline từ nghiên cứu đến sản phẩm có thể trải nghiệm thực tế, nhóm xây dựng một demo web tương tác sử dụng framework Streamlit — một thư viện Python phổ biến cho phép xây dựng nhanh các ứng dụng web cho data science và machine learning chỉ với mã Python thuần, không cần kiến thức về frontend (HTML/CSS/JavaScript). File `app.py` chứa toàn bộ logic của demo, sử dụng ONNX Runtime (không phải PyTorch) để thực hiện inference, một lựa chọn có chủ đích giúp giảm thiểu các dependency cần cài đặt

(không cần PyTorch nặng nề), tăng tốc độ khởi động ứng dụng và tính di động (portable) khi triển khai.

Demo được thiết kế với các tính năng chính sau: cho phép người dùng upload một ảnh rác thải bất kỳ ở định dạng JPG hoặc PNG thông qua giao diện kéo-thả hoặc chọn file; hiển thị song song ảnh gốc đã upload và kết quả phân loại theo thời gian thực ngay sau khi xử lý; thể hiện rõ tên lớp được dự đoán kèm theo biểu tượng emoji trực quan tương ứng giúp tăng tính thân thiện và dễ hiểu cho người dùng không chuyên; sử dụng thanh progress bar để thể hiện trực quan mức độ tin cậy (confidence) của dự đoán; cung cấp gợi ý cụ thể bằng tiếng Việt về cách xử lý đúng loại rác đó; và cuối cùng hiển thị danh sách top-3 dự đoán có xác suất cao nhất kèm theo phần trăm tương ứng, giúp người dùng hiểu rõ hơn về mức độ chắc chắn của mô hình trong các trường hợp khó phân loại.

Một ưu điểm quan trọng của demo là khả năng chạy hoàn toàn cục bộ (local) trên máy tính cá nhân, không yêu cầu kết nối internet sau khi đã hoàn tất cài đặt môi trường và các thư viện cần thiết — phù hợp cho mục đích trình diễn trong môi trường giảng đường hoặc bảo vệ đồ án mà không phụ thuộc vào chất lượng kết nối mạng. Để khởi chạy demo, chỉ cần thực hiện hai lệnh đơn giản trong terminal: kích hoạt môi trường conda phù hợp (`conda activate csc4005-dl`), sau đó chạy lệnh `streamlit run app.py`, ứng dụng sẽ tự động khởi động một server local và mở giao diện web trên trình duyệt mặc định tại địa chỉ `localhost:8501`.



Hình 4.1. Giao diện Demo Streamlit - Hệ thống phân loại rác thải

*Nguồn: Nhóm tổng hợp*

## KẾT LUẬN

Bài tập lớn đã xây dựng thành công một hệ thống phân loại rác thải hoàn chỉnh bằng ảnh sử dụng các kỹ thuật học sâu hiện đại, đạt được kết quả vượt mong đợi so với mục tiêu ban đầu đặt ra. Mô hình EfficientNet B0 fine-tune, được chọn làm best model thông qua quy trình so sánh khoa học và có kiểm soát giữa 5 cấu hình khác nhau, đạt test accuracy 97,89% và macro F1-score 97,42%, với chỉ duy nhất 12 mẫu dự đoán sai trên tổng số 758 ảnh thuộc tập test. Kết quả này không chỉ vượt xa mô hình CNN baseline (79,16%) mà còn chứng minh một cách thuyết phục về hiệu quả vượt trội của kỹ thuật transfer learning trong các bài toán phân loại ảnh có lượng dữ liệu hạn chế.

Hệ thống được xây dựng có nhiều ưu điểm nổi bật cần được nhấn mạnh. Thứ nhất, về độ chính xác, hệ thống đạt 97,89% accuracy trên 6 lớp rác thải thực tế với điều kiện ảnh đa dạng về ánh sáng và góc chụp. Thứ hai, về tính khoa học và khả năng tái lập, toàn bộ pipeline từ xử lý dữ liệu, huấn luyện, đến đánh giá đều sử dụng seed cố định và được log đầy đủ trên W&B với 5 runs có cấu hình rõ ràng. Thứ ba, về khả năng triển khai thực tế, mô hình sau khi export sang ONNX chỉ còn 0,6 MB (giảm 96% so với checkpoint PyTorch gốc) với latency trung bình chỉ 13,22 milliseconds, hoàn toàn phù hợp để tích hợp vào các hệ thống cần phản hồi thời gian thực. Thứ tư, về trải nghiệm người dùng, demo Streamlit được xây dựng với giao diện tiếng Việt trực quan, thân thiện, có gợi ý xử lý rác cụ thể.

Tuy nhiên, hệ thống vẫn còn tồn tại một số hạn chế cần được nhìn nhận một cách trung thực và khách quan. Thứ nhất, bộ dữ liệu sử dụng còn tương đối nhỏ (2.527 ảnh) và có sự mất cân bằng lớp rõ rệt, đặc biệt lớp trash chỉ có 137 ảnh, khiến mô hình có ít cơ hội học được đầy đủ sự đa dạng của lớp này. Thứ hai, một số cặp lớp có đặc trưng hình ảnh tương đồng vốn có (metal-plastic, glass-plastic, paper-cardboard) vẫn gây ra nhầm lẫn nhất định cho mô hình, dù ở tỉ lệ rất thấp — đây là giới hạn mang tính bản chất của phân loại dựa trên thông tin hình ảnh 2D. Thứ ba, hệ thống chưa được kiểm thử một cách hệ thống với dữ liệu nằm ngoài phân phối huấn luyện, ví dụ ảnh được chụp trong điều kiện thiếu sáng nghiêm trọng hoặc vật thể bị che khuất phần lớn. Thứ tư, demo hiện tại mới chỉ chạy được ở môi trường cục bộ, chưa được triển khai trên môi trường cloud công khai hay đóng gói thành ứng dụng di động.

Dựa trên những hạn chế đã nhận diện, nhóm đề xuất một số hướng phát triển cụ thể cho các nghiên cứu và phiên bản tiếp theo của hệ thống. Trước tiên, cần ưu tiên thu thập bổ sung dữ liệu, đặc biệt tập trung vào lớp trash và các cặp lớp dễ gây nhầm lẫn đã được xác định, với mục tiêu đạt tối thiểu khoảng 500 ảnh cho mỗi lớp. Tiếp theo, có thể áp dụng các kỹ thuật model compression nâng cao như lượng tử hóa INT8 (INT8 quantization) hoặc

knowledge distillation để tiếp tục giảm kích thước mô hình và tăng tốc độ inference hơn nữa. Ngoài ra, việc mở rộng số lượng lớp rác thải được phân loại (ví dụ bổ sung thêm các lớp như rác hữu cơ - organic waste, hoặc rác điện tử - e-waste) sẽ giúp hệ thống có tính ứng dụng thực tế cao hơn. Cuối cùng, nhóm hướng đến việc triển khai mô hình trên các thiết bị di động sử dụng các công nghệ như TensorFlow Lite hoặc ONNX Runtime Mobile, hoặc tích hợp trực tiếp vào các hệ thống thùng rác thông minh có tích hợp camera, từ đó đưa giải pháp công nghệ này tiến gần hơn đến việc giải quyết bài toán môi trường thực tế mà ban đầu đã thúc đẩy nhóm thực hiện đề tài này.

Quá trình thực hiện bài tập lớn đã giúp nhóm có cơ hội vận dụng toàn diện kiến thức đã học vào một bài toán thực tiễn có ý nghĩa, từ xây dựng pipeline dữ liệu, thiết kế và huấn luyện mô hình, theo dõi thí nghiệm khoa học, phân tích kết quả và lỗi, đến triển khai sản phẩm hoàn chỉnh. Đây là nền tảng quan trọng để nhóm tiếp tục phát triển kỹ năng học sâu và ứng dụng AI vào các bài toán thực tế khác trong tương lai.

## PHỤ LỤC

### **Phụ lục 1. Hình ảnh minh họa bổ sung**

Phần này trình bày các hình ảnh minh họa bổ sung không được đưa vào nội dung chính của báo cáo, bao gồm thêm các mẫu ảnh dữ liệu, kết quả Grad-CAM trên các lớp còn lại, và ảnh chụp màn hình chi tiết quá trình huấn luyện trên W&B.



```

csc4005-garbage-classification/
|   app.py
|   README.md
|   requirements.txt
|
├── configs/
|   ├── baseline.json
|   ├── resnet18_finetune.json
|   └── mobilenet_finetune.json
|
├── src/
|   ├── dataset.py
|   ├── models.py
|   ├── train.py
|   ├── utils.py
|   ├── export_onnx.py
|   ├── infer_onnx.py
|   ├── gradcam.py
|   └── error_analysis.py
|
├── outputs/
|   ├── cnn_baseline/
|   ├── resnet18_finetune/
|   ├── mobilenet_finetune/
|   ├── resnet18_head_only/
|   ├── efficientnet_finetune/
|   ├── models/
|   |   └── best_model.onnx
|   ├── figures/
|   |   ├── error_analysis/
|   |   └── gradcam/
|   └── metrics/
|       └── eval_onnx.json
|
└── reports/
    ├── experiment_log.md
    └── error_analysis.md

```

Hình .2. Ảnh minh họa bổ sung quá trình thực hiện

*Nguồn: Nhóm tổng hợp*

## Phụ lục 2. Mã nguồn và tài liệu liên quan

Toàn bộ mã nguồn, cấu hình thí nghiệm và kết quả của bài tập lớn được nhóm lưu trữ và quản lý phiên bản công khai trên GitHub, bao gồm:

- **Link GitHub Repository:** <https://github.com/Phunghuutai7125/csc4005-garbage>
- **Link W&B Project:** <https://wandb.ai/phunghuutai07122005-/csc4005-khmt16-01>
- **Cấu trúc repository chính:**
  - `src/` — Toàn bộ mã nguồn: `dataset.py`, `models.py`, `train.py`, `utils.py`, `export_onnx.py`, `infer_onnx.py`, `gradcam.py`, `error_analysis.py`.
  - `configs/` — File cấu hình JSON cho từng thí nghiệm.
  - `outputs/` — Kết quả huấn luyện: `checkpoint`, `learning curves`, `confusion matrix`, `metrics`.
  - `app.py` — Mã nguồn demo Streamlit.
  - `README.md` — Hướng dẫn cài đặt và chạy lại toàn bộ pipeline.
- **Hướng dẫn chạy lại thí nghiệm:** chi tiết được trình bày trong file `README.md` của repository, bao gồm các bước cài đặt môi trường Conda, cài thư viện từ `requirements.txt`, và lệnh chạy huấn luyện cho từng mô hình.
- **Dataset:** Garbage Classification Dataset, công khai trên Kaggle tại <https://www.kaggle.com/datasets/asdasdasdasdas/garbage-classification>.

## DANH MỤC TÀI LIỆU THAM KHẢO

### Tiếng Việt

- [1] Garbage Classification Dataset – Kaggle. <https://www.kaggle.com/datasets/asdasdasdasdas/garbage-classification>. Truy cập tháng 6/2026.

### Tiếng Anh

- [1] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.
- [2] Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 6105–6114.
- [3] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 4510–4520.
- [4] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 618–626.
- [5] Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS)*, 32.
- [6] ONNX Runtime Documentation. <https://onnxruntime.ai/docs/>. Microsoft. Truy cập tháng 6/2026.
- [7] Weights & Biases Documentation. <https://docs.wandb.ai/>. Truy cập tháng 6/2026.
- [8] Streamlit Documentation. <https://docs.streamlit.io/>. Truy cập tháng 6/2026.
- [9] Torchvision Models Documentation. <https://pytorch.org/vision/stable/models.html>. Truy cập tháng 6/2026.

- [10] Deng, J., Dong, W., Socher, R., Li, L. J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 248–255.
- [11] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.